

TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

**THIẾT KẾ VÀ PHÁT TRIỂN NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ B2C ĐA
NGÀNH HÀNG THEO KIẾN TRÚC MICROSERVICES DỰA TRÊN
RESTFUL API**

Giảng viên hướng dẫn: ThS. Trương Mạnh Đạt

Nhóm sinh viên thực hiện:

1. Lê Gia Hưng
2. Nguyễn Văn Hiếu
3. Trần Phương Thảo
4. Vũ Mạnh Tiến
5. Nguyễn Văn Tùng

HÀ NỘI - 2026

TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: PHÁT TRIỂN PHẦN MỀM HƯỚNG DỊCH VỤ

**Tên đề tài: Thiết kế và phát triển nền tảng thương mại điện tử B2C đa ngành
hàng theo kiến trúc Microservices dựa trên RESTful API.**

STT	Mã SV	Họ và tên	Điểm bằng số	Điểm bằng chữ
1	2311063231	Lê Gia Hưng		
2	2311063323	Nguyễn Văn Hiếu		
3	2311063285	Trần Phương Thảo		
4	2311063253	Vũ Mạnh Tiến		
5	2311062413	Nguyễn Văn Tùng		

GIẢNG VIÊN CHẤM 1

GIẢNG VIÊN CHẤM 2

HÀ NỘI - 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện bài tập lớn môn Phân tích hệ thống thông tin với đề tài “*Thiết kế và phát triển nền tảng thương mại điện tử B2C đa ngành hàng theo kiến trúc Microservices dựa trên RESTful API*”, nhóm em đã có cơ hội được vận dụng những kiến thức đã học, đồng thời trau dồi thêm nhiều kinh nghiệm quý báu trong lĩnh vực phân tích và thiết kế hệ thống thông tin, phát triển phần mềm và xây dựng các ứng dụng quản lý trong thực tiễn.

Để hoàn thành được bài báo cáo này, chúng em xin được bày tỏ lòng biết ơn sâu sắc nhất đến thầy giáo Trương Mạnh Đạt - người đã tận tâm hướng dẫn, luôn đồng hành và hỗ trợ em trong suốt quá trình thực hiện đề tài. Cô không chỉ truyền đạt kiến thức chuyên môn một cách tận tình, dễ hiểu, mà còn truyền cho em niềm say mê học hỏi, tinh thần trách nhiệm và sự cẩn trọng trong từng chi tiết. Sự chỉ bảo, động viên và định hướng của thầy chính là nguồn động lực lớn giúp chúng em vượt qua những khó khăn trong quá trình nghiên cứu và hoàn thiện bài tập lớn này.

Em cũng xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Trường Đại học Tài nguyên và Môi trường Hà Nội, cùng các thầy cô trong khoa Công nghệ thông tin đã tận tâm giảng dạy, tạo mọi điều kiện thuận lợi để em có thể học tập, nghiên cứu và hoàn thành bài báo cáo.

Chúng em đã dành nhiều tâm huyết, thời gian và công sức để hoàn thiện bài tập lớn này với tinh thần học hỏi nghiêm túc nhất, song do kiến thức và kinh nghiệm thực tế còn hạn chế, bài báo cáo chắc chắn vẫn không tránh khỏi những thiếu sót. Nhóm em rất mong nhận được sự thông cảm, góp ý và chỉ dẫn thêm từ cô giáo cũng như các thầy cô trong khoa để bài làm của em được hoàn thiện hơn nữa.

Chúng em xin trân trọng cảm ơn!

MỤC LỤC

MỞ ĐẦU	1
1. Lý do chọn đề tài	1
1.1 Đặt vấn đề	1
1.2 Thực trạng và yêu cầu	2
1.3 Ý nghĩa và định hướng giải pháp	2
2. Mục tiêu nghiên cứu	3
2.1 Mục tiêu tổng quát	3
2.2 Mục tiêu cụ thể	3
3. Phạm vi nghiên cứu	3
4. Phương pháp thực hiện	4
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT CÔNG NGHỆ VÀ GIẢI PHÁP	5
1.1 Tổng quan về kiến trúc và mô hình phát triển	5
1.1.1 Mô hình Client–Server dựa trên API	5
1.1.2 Kiến trúc hướng dịch vụ	6
1.1.3 Giao tiếp đồng bộ và bất đồng bộ	6
1.1.4. Kiến trúc Microservices	7
1.2. Nguyên lý thiết kế và tiêu thụ Web API	7
1.2.1. RESTful Web Service	7
1.2.2. Sử dụng phương thức HTTP	8
1.2.3. Sử dụng HTTP Status Code	8
1.3 Công nghệ sử dụng	9
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	10
2.1 Phân tích bài toán và yêu cầu hệ thống	10
2.1.1 Tác nhân	10
2.1.2 Dữ liệu cần quản lý	10
2.1.3. Yêu cầu chức năng trọng tâm	11
2.1.4 Quy tắc nghiệp vụ	11
2.1.5. Yêu cầu phi chức năng	12
2.2. Mô hình hóa chức năng và nghiệp vụ	12
2.2.1 Sơ đồ Use Case	13
2.2.2 Đặc tả Use Case	13
2.2.3 Biểu đồ hoạt động và biểu đồ tuần tự	13
2.3 Thiết kế kiến trúc hệ thống	13

2.3.1. Kiến trúc tổng thể.....	14
2.3.2. Giao tiếp và triển khai các thành phần.....	16
2.3.3. Tổ chức các tầng bên trong Backend	16
2.4. Thiết kế dữ liệu.....	18
2.4.1. Các nhóm thực thể chính	18
2.4.2. Các thực thể và quan hệ cốt lõi.....	18
2.4.3. Sơ đồ ERD	19
2.4.4. Biểu đồ lớp	20
2.5 Thiết kế API dịch vụ	20
2.5.1 Quy ước API.....	20
2.5.2 Các API chính	21
2.5.3 Phản hồi và xử lý lỗi.....	21
2.6 Thiết kế ứng dụng client.....	22
2.6.1 Thiết kế giao diện	22
2.6.2 Thiết kế tích hợp API	22
2.6.3 Luồng dữ liệu và trạng thái.....	23
CHƯƠNG 3. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG	24
3.1 Môi trường và công cụ phát triển	24
3.1.1. Môi trường phát triển	24
3.1.2. Môi trường phát triển backend.....	24
3.1.3. Công cụ kiểm thử	25
3.2 Cài đặt Backend Service API	25
3.2.1 Cấu trúc dự án Backend	25
3.2.2 Hiện thực dữ liệu và truy cập cơ sở dữ liệu	27
3.2.3 Hiện thực API chính.....	28
3.2.4 Hiện thực xử lý nghiệp vụ.....	30
3.3 Cài đặt Client Application	32
3.3.1 Cấu trúc dự án Client.....	32
3.3.2 Giao diện và chức năng chính	34
3.3.3 Tích hợp Client với Backend API.....	34
3.3.4 Xử lý trạng thái và lỗi.....	35
3.4 Triển khai hệ thống.....	35
3.4.1. Mô hình triển khai cục bộ.....	35
3.4.2. Thứ tự khởi động.....	36
3.4.3. Cồng chạy các thành phần.....	37
CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG	38

4.1 Kiểm thử API.....	38
4.1.1 Kiểm thử Danh mục sản phẩm	38
4.1.2 Kiểm thử Đơn hàng	38
4.1.3 Kiểm thử Thanh toán.....	38
4.2 Kiểm thử tích hợp Client API	39
4.3 Đánh giá mức độ đáp ứng.....	39
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
1. Kết luận.....	41
2. Hạn chế.....	41
3. Hướng phát triển.....	41
TÀI LIỆU THAM KHẢO.....	42

MỞ ĐẦU

1. Lý do chọn đề tài

1.1 Đặt vấn đề

Sự phát triển của thương mại điện tử làm cho một hệ thống bán hàng trực tuyến ngày nay không còn đơn thuần là nơi hiển thị sản phẩm và tiếp nhận đơn đặt hàng. Đằng sau một thao tác tưởng như đơn giản của khách hàng, chẳng hạn chọn sản phẩm, áp dụng mã giảm giá và tiến hành thanh toán, là sự phối hợp của nhiều nhóm nghiệp vụ có tính chất khác nhau: quản lý tài khoản và quyền truy cập, danh mục và biến thể sản phẩm, tồn kho, giỏ hàng, khuyến mãi, đặt hàng, tính phí giao hàng, thanh toán và tương tác sau mua.

Khi các chức năng này được phát triển trong một khối ứng dụng có mức độ phụ thuộc cao, thay đổi ở một nghiệp vụ có thể kéo theo ảnh hưởng đến nhiều phần khác của hệ thống. Điều này đặc biệt rõ khi dự án có nhiều thành viên cùng phát triển: phạm vi mã nguồn dễ chồng lấn, trách nhiệm giữa các thành phần khó xác định, lỗi khó cô lập và quá trình tích hợp trở nên phức tạp. Vấn đề vì thế không chỉ nằm ở việc hệ thống có thực hiện được chức năng bán hàng hay không, mà còn nằm ở cách tổ chức kiến trúc để mỗi miền nghiệp vụ có ranh giới rõ ràng, dữ liệu có chủ sở hữu xác định và các thành phần vẫn có thể phối hợp thành một quy trình mua hàng thống nhất.

Từ thực tế đó, đề tài “Thiết kế và phát triển nền tảng thương mại điện tử B2C đa ngành hàng theo kiến trúc Microservices dựa trên RESTful API” được lựa chọn nhằm nghiên cứu và hiện thực một hệ thống thương mại điện tử theo hướng phân tách trách nhiệm. DynamicMart được xác định là mô hình B2C trong đó một doanh nghiệp trực tiếp cung cấp hàng hóa cho nhiều khách hàng; hệ thống không mở rộng sang mô hình marketplace có nhiều người bán độc lập. Kiến trúc hiện hành được chia thành sáu dịch vụ nghiệp vụ gồm tài khoản, danh mục sản phẩm, giỏ hàng và khuyến mãi, đơn hàng, thanh toán và tính phí giao hàng, tương tác khách hàng và báo cáo; phía trước các dịch vụ là một API Gateway đóng vai trò điểm truy cập thống nhất.

Việc lựa chọn Microservices trong đề tài không xuất phát từ mục tiêu chia nhỏ hệ thống một cách hình thức, mà từ nhu cầu tạo ra ranh giới trách nhiệm rõ ràng giữa các miền nghiệp vụ. Mỗi dịch vụ cần có khả năng phát triển, kiểm thử và thay đổi tương

đối độc lập, đồng thời phải tuân thủ một hợp đồng giao tiếp chung để toàn hệ thống vẫn hoạt động như một sản phẩm thống nhất.

1.2 Thực trạng và yêu cầu

Trong quá trình mua hàng trực tuyến, nhiều thông tin như giá sản phẩm, số lượng còn lại, mức giảm giá, phí giao hàng hay trạng thái thanh toán có thể thay đổi liên tục. Vì vậy, hệ thống cần kiểm tra và xử lý các thông tin này ở phía máy chủ thay vì hoàn toàn phụ thuộc vào dữ liệu từ giao diện người dùng.

Ngoài ra, hệ thống cần hạn chế các trường hợp như tạo đơn hàng nhiều lần do người dùng thao tác lặp, sử dụng mã giảm giá không hợp lệ, đặt mua sản phẩm đã hết hàng hoặc ghi nhận sai trạng thái thanh toán. Khách hàng cũng chỉ được phép xem và thao tác trên thông tin thuộc tài khoản của mình.

Do đó, yêu cầu đặt ra không chỉ là xây dựng đầy đủ chức năng bán hàng mà còn phải bảo đảm quá trình từ chọn sản phẩm, đặt hàng đến thanh toán diễn ra chính xác, an toàn và ổn định.

1.3 Ý nghĩa và định hướng giải pháp

Đề tài lựa chọn kiến trúc Microservices để phân chia hệ thống thành các nhóm chức năng tương đối độc lập như tài khoản, sản phẩm, giỏ hàng, đơn hàng, thanh toán và tương tác khách hàng. Cách tổ chức này giúp từng thành viên có thể phát triển và kiểm thử phần việc của mình rõ ràng hơn, đồng thời hạn chế ảnh hưởng giữa các chức năng khi hệ thống được thay đổi hoặc mở rộng.

Các dịch vụ trao đổi dữ liệu thông qua RESTful API, trong khi API Gateway đóng vai trò là điểm tiếp nhận chung giữa giao diện và hệ thống phía máy chủ. Một số thông tin cần xử lý giữa nhiều dịch vụ có thể được trao đổi thông qua cơ chế gửi nhận sự kiện.

Thông qua đề tài, nhóm có cơ hội vận dụng kiến thức về phát triển web, thiết kế hệ thống, xây dựng API, xác thực người dùng, xử lý đơn hàng và tích hợp các dịch vụ thực tế vào một hệ thống thương mại điện tử hoàn chỉnh.

2. Mục tiêu nghiên cứu

2.1 Mục tiêu tổng quát

Thiết kế và xây dựng nền tảng thương mại điện tử B2C đa ngành hàng theo kiến trúc Microservices dựa trên RESTful API, đáp ứng các chức năng cơ bản của quá trình mua bán trực tuyến và tạo nền tảng thuận lợi cho việc phát triển, kiểm thử và mở rộng hệ thống.

2.2 Mục tiêu cụ thể

Xây dựng hệ thống xác thực và phân quyền người dùng, đồng thời sử dụng API Gateway làm điểm truy cập chung cho ứng dụng.

Phân chia các nhóm chức năng chính như tài khoản, sản phẩm, giỏ hàng, khuyến mãi, đơn hàng, thanh toán và tương tác khách hàng thành các thành phần có trách nhiệm rõ ràng.

Xây dựng các chức năng phục vụ quá trình mua hàng như tìm kiếm sản phẩm, quản lý giỏ hàng, áp dụng khuyến mãi, đặt hàng, tính phí giao hàng và thanh toán.

Xây dựng giao diện bằng Next.js, bảo đảm người dùng có thể thực hiện các thao tác mua hàng một cách thuận tiện và nhận được phản hồi rõ ràng khi tải dữ liệu, xảy ra lỗi hoặc hoàn thành thao tác.

Kiểm thử các chức năng và luồng nghiệp vụ chính nhằm đánh giá mức độ hoạt động ổn định và đúng yêu cầu của hệ thống.

3. Phạm vi nghiên cứu

Đề tài tập trung xây dựng nền tảng thương mại điện tử theo mô hình B2C, trong đó một doanh nghiệp trực tiếp cung cấp nhiều loại sản phẩm cho khách hàng.

Phạm vi chính gồm quản lý tài khoản và địa chỉ, quản lý danh mục và sản phẩm, quản lý tồn kho, giỏ hàng, khuyến mãi, đặt hàng, tính phí giao hàng, thanh toán và các giao diện dành cho khách hàng và quản trị viên.

Hệ thống hỗ trợ các hình thức thanh toán như COD, VNPay và một số phương thức thanh toán trực tuyến khác phục vụ quá trình thử nghiệm và tích hợp. GHN được sử dụng để hỗ trợ lựa chọn địa chỉ và tính phí giao hàng.

Đề tài chưa tập trung vào các chức năng mở rộng như mô hình nhiều người bán, nhiều kho hàng, chương trình khách hàng thân thiết, hoàn tiền tự động, quản lý vận đơn

và theo dõi giao hàng chi tiết hoặc triển khai hệ thống trong môi trường thực tế quy mô lớn.

4. Phương pháp thực hiện

Quá trình thực hiện bắt đầu bằng việc khảo sát yêu cầu của hệ thống thương mại điện tử và xác định phạm vi nghiệp vụ. Ba use case trọng tâm gồm Danh mục sản phẩm, Đơn hàng và Thanh toán; các chức năng tài khoản, giỏ hàng, voucher và vận chuyển đóng vai trò hỗ trợ cho ba luồng này.

Kiến trúc, hợp đồng API, mô hình dữ liệu và quy tắc nghiệp vụ được xác định trước khi hiện thực. Các thành phần được phát triển theo ranh giới dịch vụ và tích hợp thông qua API Gateway hoặc sự kiện.

Trong quá trình phát triển, frontend được xây dựng bằng Next.js và kết nối với các dịch vụ phía máy chủ thông qua API Gateway. Các chức năng được kiểm tra riêng trước khi tích hợp thành các luồng hoàn chỉnh như đăng nhập, lựa chọn sản phẩm, thêm vào giỏ hàng, đặt hàng và thanh toán.

Cuối cùng, các API, giao diện và luồng nghiệp vụ được kiểm thử, sửa lỗi và đối chiếu với yêu cầu ban đầu để đánh giá mức độ hoàn thiện.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT CÔNG NGHỆ VÀ GIẢI PHÁP

1.1 Tổng quan về kiến trúc và mô hình phát triển

Chương này trình bày các mô hình kiến trúc, nguyên lý thiết kế Web API và những công nghệ làm nền tảng cho hệ thống DynamicMart. Nội dung tập trung vào khái niệm, vai trò và lý do lựa chọn giải pháp; cấu trúc mã nguồn và quá trình hiện thực cụ thể được trình bày ở các chương sau.

1.1.1 Mô hình Client–Server dựa trên API

Client–Server là mô hình trong đó hệ thống được chia thành hai nhóm thành phần chính. Client cung cấp giao diện, tiếp nhận thao tác và gửi yêu cầu; Server tiếp nhận yêu cầu, xử lý nghiệp vụ, truy cập dữ liệu và trả kết quả.

Trong mô hình này, Client không truy cập trực tiếp cơ sở dữ liệu. Mọi thao tác liên quan đến tài khoản, sản phẩm, giỏ hàng, đơn hàng hoặc thanh toán đều được thực hiện thông qua API do backend cung cấp.

Mô hình Client–Server giúp phân tách trách nhiệm rõ ràng:

- Client tập trung vào giao diện và trải nghiệm sử dụng.
- Backend tập trung vào quy tắc nghiệp vụ và tính toàn vẹn của dữ liệu.
- Cơ sở dữ liệu được bảo vệ phía sau backend.
- Client và backend có thể được phát triển, kiểm thử và triển khai tương đối độc lập.

DynamicMart áp dụng mô hình này với ứng dụng web làm Client và các dịch vụ backend làm Server. Hai phía trao đổi dữ liệu chủ yếu thông qua HTTP/HTTPS và định dạng JSON.

Kiến trúc dựa trên API tổ chức việc giao tiếp giữa các thành phần thông qua những hợp đồng đã được xác định. Hợp đồng API quy định địa chỉ tài nguyên, phương thức HTTP, dữ liệu gửi lên, cấu trúc phản hồi và cách biểu diễn lỗi.

API giữ vai trò trung gian giữa Client và backend. Client không cần biết dữ liệu được lưu trong bảng nào hoặc nghiệp vụ được xử lý bởi lớp nào. Client chỉ cần gửi request đúng hợp đồng và xử lý response nhận được.

Việc sử dụng API đem lại một số lợi ích:

- Giảm sự phụ thuộc trực tiếp giữa giao diện và cách cài đặt backend.
- Cho phép thay đổi giao diện mà không làm thay đổi cơ sở dữ liệu.
- Tạo điều kiện sử dụng chung backend cho nhiều loại Client.

- Thuận lợi cho kiểm thử, phân quyền và quản lý phiên bản.
- Giúp các dịch vụ trao đổi dữ liệu theo cấu trúc thống nhất.

Trong DynamicMart, API được sử dụng cho cả giao tiếp giữa Client với backend và một số giao tiếp đồng bộ, bất đồng bộ giữa các dịch vụ.

1.1.2 Kiến trúc hướng dịch vụ

Kiến trúc hướng dịch vụ tổ chức hệ thống thành các đơn vị phần mềm tương ứng với từng phạm vi nghiệp vụ. Mỗi dịch vụ cung cấp chức năng thông qua một hợp đồng giao tiếp rõ ràng và hạn chế phụ thuộc vào cách cài đặt nội bộ của dịch vụ khác.

Một dịch vụ thường có các đặc điểm:

- Phụ trách một nhóm nghiệp vụ xác định.
- Che giấu dữ liệu và logic xử lý nội bộ.
- Cung cấp chức năng thông qua API hoặc sự kiện.
- Có thể được phát triển và kiểm thử tương đối độc lập.
- Không truy cập trực tiếp dữ liệu thuộc quyền sở hữu của dịch vụ khác.

Cách tổ chức này phù hợp với những hệ thống có nhiều nhóm nghiệp vụ như tài khoản, sản phẩm, giỏ hàng, đơn hàng và thanh toán. Mỗi nhóm có dữ liệu, quy tắc và tốc độ thay đổi khác nhau nên cần được phân chia trách nhiệm rõ ràng.

1.1.3 Giao tiếp đồng bộ và bất đồng bộ

Các thành phần trong hệ thống có thể giao tiếp theo hai cơ chế: đồng bộ và bất đồng bộ.

Giao tiếp đồng bộ được sử dụng khi thành phần gửi yêu cầu cần nhận kết quả ngay để tiếp tục xử lý. REST API là hình thức giao tiếp đồng bộ chính trong hệ thống. Ví dụ, việc lấy thông tin sản phẩm hoặc kiểm tra địa chỉ cần trả kết quả trực tiếp cho bước tiếp theo.

Giao tiếp bất đồng bộ được sử dụng khi một nghiệp vụ có thể thông báo kết quả thông qua sự kiện mà không yêu cầu dịch vụ nhận phải xử lý cùng thời điểm. Message Queue giúp lưu giữ và chuyển sự kiện giữa các dịch vụ.

Việc kết hợp hai cơ chế đem lại sự cân bằng:

- REST phù hợp với truy vấn và xử lý cần phản hồi ngay.
- Message Queue giúp giảm phụ thuộc thời gian giữa các dịch vụ.
- Sự kiện hỗ trợ retry khi dịch vụ nhận tạm thời chưa sẵn sàng.

- Các nghiệp vụ dài có thể được chia thành nhiều bước xử lý.

DynamicMart sử dụng REST API cho những thao tác cần phản hồi trực tiếp và RabbitMQ cho các sự kiện nghiệp vụ phù hợp với xử lý bất đồng bộ.

1.1.4. Kiến trúc Microservices

Microservices là một cách triển khai kiến trúc hướng dịch vụ, trong đó hệ thống được chia thành nhiều ứng dụng có phạm vi nhỏ và tập trung vào từng miền nghiệp vụ. Mỗi dịch vụ có thể sở hữu dữ liệu, cấu hình và vòng đời triển khai riêng.

Không phải mọi dự án Spring Boot có nhiều package đều là Microservices. Để được xem là kiến trúc Microservices, các thành phần cần có ranh giới nghiệp vụ tương đối độc lập, giao tiếp qua hợp đồng và không dùng chung trực tiếp tầng dữ liệu nội bộ.

DynamicMart áp dụng kiến trúc hướng dịch vụ theo định hướng Microservices vì:

- Các nhóm nghiệp vụ được triển khai thành những dịch vụ riêng.
- Mỗi dịch vụ sở hữu cơ sở dữ liệu thuộc phạm vi của mình.
- Client truy cập hệ thống thông qua một cổng API thống nhất.
- Các dịch vụ giao tiếp bằng REST API hoặc sự kiện.
- Một dịch vụ không truy vấn trực tiếp cơ sở dữ liệu của dịch vụ khác.

Kiến trúc này giúp hệ thống thể hiện rõ tinh thần của phát triển phần mềm hướng dịch vụ. Tuy nhiên, nó cũng làm tăng yêu cầu về cấu hình, kiểm thử tích hợp, xử lý lỗi và duy trì tính nhất quán giữa các dịch vụ.

1.2. Nguyên lý thiết kế và tiêu thụ Web API

1.2.1. RESTful Web Service

REST là phong cách kiến trúc trong đó dữ liệu và chức năng được biểu diễn dưới dạng tài nguyên. Mỗi tài nguyên được xác định bằng một URI và được thao tác thông qua các phương thức HTTP.

RESTful API không đặt trọng tâm vào tên hành động trong endpoint mà tập trung vào tài nguyên. Chẳng hạn, sản phẩm, giỏ hàng, địa chỉ và đơn hàng được biểu diễn thành những nhóm tài nguyên riêng.

Các nguyên tắc REST được áp dụng gồm:

- Tài nguyên được xác định bằng URI rõ ràng.
- Endpoint ưu tiên sử dụng danh từ

- Phương thức HTTP thể hiện loại thao tác.
- Request chứa đủ thông tin để backend xử lý.
- Response sử dụng status code phù hợp.
- Dữ liệu trao đổi được biểu diễn thống nhất bằng JSON.
- Client và backend được tách biệt về trách nhiệm.

1.2.2. Sử dụng phương thức HTTP

Các phương thức HTTP được sử dụng theo ý nghĩa của thao tác:

Phương thức	Mục đích
GET	Truy vấn một tài nguyên hoặc danh sách tài nguyên
POST	Tạo tài nguyên hoặc thực hiện một nghiệp vụ
PUT	Cập nhật đầy đủ thông tin của tài nguyên
PATCH	Cập nhật một phần hoặc thay đổi trạng thái
DELETE	Xóa hoặc vô hiệu hóa tài nguyên

Việc sử dụng đúng phương thức giúp hợp đồng API dễ hiểu và giảm tình trạng endpoint được đặt tên theo hành động không nhất quán.

1.2.3. Sử dụng HTTP Status Code

Status code phản ánh kết quả tổng quát của request:

Status Code	Ý nghĩa sử dụng
200 OK	Yêu cầu được xử lý thành công
201 Created	Tài nguyên được tạo thành công
204 No Content	Xử lý thành công nhưng không có nội dung trả về
400 Bad Request	Dữ liệu hoặc tham số không hợp lệ
401 Unauthorized	Chưa xác thực hoặc phiên không còn hợp lệ
403 Forbidden	Đã xác thực nhưng không có quyền
404 Not Found	Không tìm thấy tài nguyên
409 Conflict	Xung đột trạng thái hoặc dữ liệu
422 Unprocessable Entity	Dữ liệu đúng định dạng nhưng vi phạm nghiệp vụ
500 Internal Server Error	Lỗi không dự kiến tại máy chủ
503 Service Unavailable	Dịch vụ tạm thời chưa sẵn sàng

Client không chỉ dựa vào nội dung thông báo mà còn sử dụng status code và mã lỗi để lựa chọn cách phản hồi phù hợp.

1.3 Công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Backend	Java, Spring Boot	Xây dựng dịch vụ, REST API và xử lý nghiệp vụ
API Gateway	Spring Cloud Gateway	Xác thực và định tuyến request
Truy cập dữ liệu	Spring Data JPA	Ánh xạ Entity và thao tác với cơ sở dữ liệu
Cơ sở dữ liệu	PostgreSQL	Lưu trữ dữ liệu của các dịch vụ
Quản lý migration	Flyway	Quản lý phiên bản cấu trúc dữ liệu
Giao tiếp sự kiện	RabbitMQ	Truyền sự kiện bất đồng bộ
Xác thực	Spring Security, JWT	Xác thực và phân quyền API
Client	Next.js, React	Xây dựng ứng dụng web và tiêu thụ API
Ngôn ngữ Client	TypeScript	Kiểm tra kiểu và giảm lỗi khi tích hợp
Trình bày giao diện	Tailwind CSS	Xây dựng bố cục và giao diện đáp ứng
Quản lý backend	Maven	Quản lý thư viện và quá trình build
Quản lý Client	npm	Quản lý thư viện frontend
Kiểm thử API	Postman, Swagger	Gửi request và kiểm tra hợp đồng API
Quản lý mã nguồn	Git	Theo dõi thay đổi và phối hợp phát triển
Đóng gói	Docker	Chuẩn hóa môi trường chạy
Trợ lý AI	Python, FastAPI, Ollama	Cung cấp API hỏi đáp và sinh câu trả lời từ ngữ cảnh truy xuất
Cơ sở dữ liệu vector	Qdrant	Lưu vector và truy xuất nguồn tri thức theo tenant

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích bài toán và yêu cầu hệ thống

DynamicMart được xây dựng nhằm hỗ trợ quá trình mua bán trực tuyến, từ khi khách hàng tìm kiếm sản phẩm đến khi hoàn tất đặt hàng và thanh toán. Hệ thống cần cung cấp trải nghiệm thuận tiện cho người mua, đồng thời giúp quản trị viên kiểm soát sản phẩm, tồn kho, khuyến mãi và các giao dịch phát sinh. Bên cạnh các chức năng thông thường, hệ thống phải bảo đảm thông tin về giá, số lượng tồn và trạng thái đơn hàng được xử lý chính xác trong suốt quá trình mua hàng.

2.1.1 Tác nhân

Tác nhân	Nhu cầu chính
Khách hàng	Quản lý thông tin cá nhân và địa chỉ nhận hàng; thêm, cập nhật hoặc xóa sản phẩm trong giỏ hàng; sử dụng mã giảm giá; kiểm tra thông tin đơn hàng; lựa chọn phương thức thanh toán; đặt hàng và xem kết quả thanh toán.
Quản trị viên	Quản lý người dùng, danh mục, sản phẩm, biến thể, hình ảnh, số lượng tồn kho và chương trình khuyến mãi; theo dõi giao dịch thanh toán và tình trạng hoạt động của các chức năng quản trị.
Cổng thanh toán	Tiếp nhận yêu cầu thanh toán, xử lý giao dịch và gửi kết quả về hệ thống.
Dịch vụ giao hàng	Cung cấp thông tin địa giới và ước tính phí vận chuyển theo địa chỉ nhận hàng.

2.1.2 Dữ liệu cần quản lý

Dữ liệu được tổ chức theo ba phạm vi nghiệp vụ chính và một số dữ liệu hỗ trợ:

- Danh mục sản phẩm: danh mục, thuộc tính, sản phẩm, biến thể, hình ảnh, giá bán, tồn kho và nguồn tri thức phục vụ trợ lý mua sắm.

- Đơn hàng: giỏ hàng được chọn, phiên đặt hàng, địa chỉ giao hàng, voucher, phí vận chuyển, các dòng hàng và lịch sử trạng thái.

- Thanh toán: nghĩa vụ thanh toán, các lần thực hiện, kết quả phản hồi, phương thức COD hoặc VNPay và trạng thái giao dịch.

- Dữ liệu hỗ trợ: tài khoản, phiên đăng nhập, địa chỉ cá nhân và các bản ghi sự kiện cần thiết cho xác thực, truy vết và tích hợp.

Thông tin đã ghi nhận trong đơn hàng được lưu dưới dạng bản chụp. Việc thay đổi sản phẩm, giá bán hoặc địa chỉ sau đó không làm thay đổi nội dung giao dịch đã phát sinh.

2.1.3. Yêu cầu chức năng trọng tâm

Phạm vi phân tích và kiểm chứng tập trung vào ba use case chính:

- Danh mục sản phẩm: quản lý danh mục, sản phẩm, biến thể, hình ảnh và tồn kho; hỗ trợ duyệt, tìm kiếm, lọc, xem chi tiết và hỏi đáp qua trợ lý mua sắm.

- Đơn hàng: chuẩn bị dữ liệu đặt hàng, kiểm tra địa chỉ, voucher, vận chuyển và tồn kho; tạo đơn, lưu bản chụp và theo dõi trạng thái.

- Thanh toán: lựa chọn COD hoặc VNPay, khởi tạo giao dịch, xác minh kết quả và cung cấp trạng thái thanh toán cho đơn hàng.

Tài khoản, giỏ hàng, voucher và vận chuyển không được tách thành use case trọng tâm; các chức năng này cung cấp dữ liệu và điều kiện cần thiết cho quá trình đặt hàng và thanh toán.

2.1.4 Quy tắc nghiệp vụ

Các quy tắc nghiệp vụ quan trọng gồm:

- Khách hàng không được đặt số lượng sản phẩm vượt quá số lượng hiện còn trong kho.

- Giá bán, mức giảm, phí vận chuyển và tổng tiền phải do hệ thống tính toán; dữ liệu gửi từ giao diện người dùng chỉ mang tính chất lựa chọn.

- Một đơn hàng chỉ được tạo khi có đầy đủ sản phẩm hợp lệ, địa chỉ nhận hàng, phương thức thanh toán và các thông tin bắt buộc khác.

- Địa chỉ sử dụng để đặt hàng phải thuộc tài khoản của khách hàng đang thực hiện giao dịch.

- Mã giảm giá chỉ được áp dụng khi còn hiệu lực, còn lượt sử dụng và đơn hàng đáp ứng đầy đủ điều kiện của chương trình.

- Sản phẩm và mã giảm giá chỉ được giữ trong một khoảng thời gian nhất định trong quá trình đặt hàng. Khi hết thời gian hoặc giao dịch thất bại, phần đã giữ phải được hoàn lại.
- Mỗi yêu cầu đặt hàng chỉ được tạo ra một đơn hàng, kể cả khi khách hàng nhấn nút xác nhận nhiều lần hoặc hệ thống nhận lại cùng một kết quả thanh toán.
- Đơn hàng thanh toán trực tuyến chỉ được xác nhận thanh toán thành công khi kết quả trả về hợp lệ và số tiền khớp với đơn hàng.
- Khách hàng chỉ được xem và chỉnh sửa dữ liệu thuộc tài khoản của mình. Quản trị viên chỉ được thực hiện các chức năng phù hợp với quyền được cấp.
- Những thông tin đã ghi nhận trong đơn hàng, như tên sản phẩm, giá bán, địa chỉ và phí giao hàng, phải được giữ nguyên để bảo đảm tính chính xác của lịch sử giao dịch.

Các quy tắc trên tạo thành logic nghiệp vụ cốt lõi. Trợ lý mua sắm chỉ trả lời từ nguồn tri thức đã kiểm duyệt, không tự tính giá, tồn kho, tổng tiền, trạng thái đơn hoặc trạng thái thanh toán; các giá trị quyết định luôn được lấy từ API nghiệp vụ.

2.1.5. Yêu cầu phi chức năng

Ngoài các chức năng nghiệp vụ, hệ thống cần đáp ứng những yêu cầu sau:

- Tính chính xác: giá trị đơn hàng, số lượng tồn kho, voucher và trạng thái thanh toán phải được xử lý nhất quán.
- Bảo mật: thông tin tài khoản và giao dịch phải được bảo vệ; người dùng chỉ được truy cập những chức năng và dữ liệu thuộc phạm vi quyền hạn.
- Khả năng truy vết: các thay đổi quan trọng liên quan đến đơn hàng và thanh toán cần được ghi nhận để hỗ trợ kiểm tra khi xảy ra sai lệch.
- Hiệu năng: các thao tác thường dùng như xem sản phẩm, cập nhật giỏ hàng và kiểm tra đơn hàng cần có thời gian phản hồi phù hợp.
- Khả năng sử dụng: giao diện cần hiển thị tốt trên nhiều kích thước màn hình và thông báo rõ trạng thái đang xử lý, không có dữ liệu hoặc xảy ra lỗi.
- Khả năng mở rộng và bảo trì: các nhóm chức năng cần được tổ chức rõ ràng để có thể sửa đổi hoặc bổ sung mà hạn chế ảnh hưởng đến phần còn lại của hệ thống.

2.2. Mô hình hóa chức năng và nghiệp vụ

Phần mô hình hóa chỉ sử dụng ba use case trọng tâm. Những thao tác hỗ trợ được đặt trong luồng của use case tương ứng để tránh chia nhỏ nghiệp vụ thành danh sách CRUD.

2.2.1 Sơ đồ Use Case

Sơ đồ thể hiện khách hàng và quản trị viên tương tác với ba use case Danh mục sản phẩm, Đơn hàng và Thanh toán. Công thanh toán và đơn vị vận chuyển chỉ tham gia tại các bước tích hợp tương ứng.

[CHÈN HÌNH 2.1: Sơ đồ Use Case tổng quát gồm Danh mục sản phẩm, Đơn hàng và Thanh toán]

2.2.2 Đặc tả Use Case

Ba use case được đặc tả ở mức đủ để xác định tác nhân, điều kiện, luồng xử lý và kết quả cần kiểm chứng.

Mã và Use Case	Tác nhân và điều kiện	Luồng chính	Ngoại lệ và kết quả
UC0 1 Danh mục sản phẩm	Khách truy cập, khách hàng, quản trị viên. Dữ liệu danh mục đã được công bố.	Duyệt hoặc quản lý danh mục; tìm kiếm, lọc, chọn biến thể, xem giá và tồn kho. Trợ lý hỗ trợ giải đáp theo nguồn đã kiểm duyệt.	Điều kiện lọc không hợp lệ hoặc không có dữ liệu được phản hồi rõ. Kết quả hiển thị đúng dữ liệu hiện hành; trợ lý không suy đoán khi thiếu nguồn.
UC0 2 Đơn hàng	Khách hàng đã đăng nhập; mặt hàng, địa chỉ và phiên đặt hàng còn hợp lệ.	Tạo phiên, chọn địa chỉ và voucher, lấy bản xem trước, kiểm tra lại dữ liệu, giữ tài nguyên và tạo đơn bằng khóa chống gửi trùng.	Hết hàng, voucher hoặc báo giá hết hạn làm dừng luồng và hoàn phần đã giữ. Một yêu cầu hợp lệ chỉ tạo một đơn hàng.
UC0 3 Thanh toán	Đơn hàng đã tạo và có nghĩa vụ thanh toán hợp lệ.	Chọn COD hoặc VNPay; khởi tạo lần thanh toán; xác minh phản hồi và truy vấn trạng thái theo đơn.	Sai chữ ký, sai số tiền hoặc phản hồi trùng không làm thay đổi giao dịch sai lệch. Kết quả hợp lệ được ghi nhận đúng một lần.

2.2.3 Biểu đồ hoạt động và biểu đồ tuần tự

Các biểu đồ chi tiết bám theo đúng ba use case trên và sử dụng cùng tên gọi trong toàn bộ báo cáo.

[CHÈN HÌNH 2.2: Biểu đồ hoạt động và tuần tự của Use Case Danh mục sản phẩm]

[CHÈN HÌNH 2.3: Biểu đồ hoạt động và tuần tự của Use Case Đơn hàng]

[CHÈN HÌNH 2.4: Biểu đồ hoạt động và tuần tự của Use Case Thanh toán]

2.3 Thiết kế kiến trúc hệ thống

DynamicMart được thiết kế theo kiến trúc hướng dịch vụ. Thay vì tập trung toàn bộ chức năng vào một ứng dụng duy nhất, hệ thống được chia thành các thành phần độc lập theo từng nhóm nghiệp vụ như tài khoản, sản phẩm, giỏ hàng, đơn hàng và thanh

toán. Cách tổ chức này giúp xác định rõ trách nhiệm của mỗi thành phần, hạn chế sự phụ thuộc giữa các chức năng và thuận tiện hơn khi bảo trì hoặc mở rộng hệ thống.

2.3.1. Kiến trúc tổng thể

Kiến trúc DynamicMart gồm ứng dụng khách, API Gateway, sáu dịch vụ nghiệp vụ, dịch vụ trợ lý AI, các kho dữ liệu và những hệ thống tích hợp bên ngoài.

Ứng dụng khách được xây dựng bằng Next.js, cung cấp giao diện cho khách hàng và quản trị viên. Thành phần này đảm nhận việc hiển thị dữ liệu, tiếp nhận thao tác của người dùng và gửi yêu cầu đến hệ thống. Các quy tắc quan trọng liên quan đến giá bán, tồn kho, voucher, đơn hàng và thanh toán vẫn được kiểm tra tại Backend để bảo đảm tính chính xác.

API Gateway là điểm tiếp nhận chung cho các yêu cầu từ ứng dụng khách. Gateway xác định dịch vụ cần xử lý, chuyển tiếp yêu cầu và kiểm tra thông tin xác thực đối với các chức năng được bảo vệ. Nhờ đó, Client không cần giao tiếp trực tiếp với từng dịch vụ riêng lẻ.

Phần nghiệp vụ cốt lõi gồm sáu dịch vụ, bên cạnh một dịch vụ trợ lý AI hỗ trợ tra cứu:

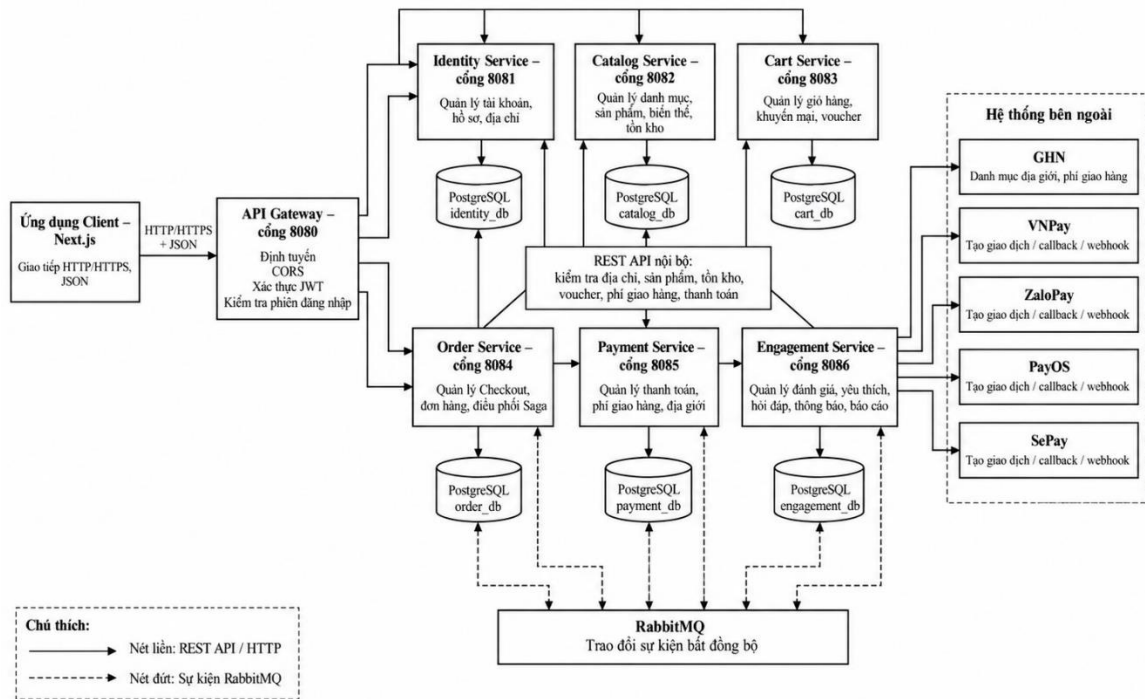
- Quản lý tài khoản, đăng nhập, hồ sơ và địa chỉ.
- Quản lý danh mục, sản phẩm, biến thể, hình ảnh và tồn kho.
- Quản lý giỏ hàng, khuyến mãi, voucher và thông tin chuẩn bị đặt hàng.
- Quản lý phiên đặt hàng, tạo đơn và trạng thái đơn hàng.
- Xử lý thanh toán, kết quả giao dịch và phí vận chuyển.
- Quản lý đánh giá, danh sách yêu thích và thông báo.

Hỗ trợ hỏi đáp bằng RAG từ nguồn tri thức đã kiểm duyệt.

Mỗi dịch vụ quản lý một cơ sở dữ liệu PostgreSQL riêng. Dịch vụ chỉ trực tiếp thay đổi dữ liệu thuộc phạm vi nghiệp vụ của mình; khi cần thông tin từ dịch vụ khác, hệ thống thực hiện giao tiếp thông qua API hoặc sự kiện. Cách tổ chức này giúp dữ liệu có chủ sở hữu rõ ràng và tránh việc nhiều dịch vụ cùng thay đổi một bảng dữ liệu.

RabbitMQ truyền các sự kiện không yêu cầu phản hồi tức thời. DynamicMart còn kết nối với GHN, cổng thanh toán và dịch vụ trợ lý AI. Trợ lý sử dụng FastAPI, Ollama và Qdrant để truy xuất nguồn tri thức; mọi dữ liệu quyết định về sản phẩm, đơn hàng và thanh toán vẫn do các dịch vụ nghiệp vụ cung cấp.

Sơ đồ kiến trúc tổng thể hệ thống thương mại điện tử DynamicMart theo kiến trúc Microservices



Thành phần	Trách nhiệm
Next.js Client	Cung cấp giao diện cửa hàng, tài khoản, giỏ hàng, đặt hàng, thanh toán và quản trị; tiếp nhận thao tác và hiển thị kết quả cho người dùng.
API Gateway	Tiếp nhận yêu cầu từ Client, định tuyến đến dịch vụ phù hợp và kiểm tra thông tin xác thực đối với các tài nguyên được bảo vệ.
Identity Service	Quản lý tài khoản, đăng nhập, phiên làm việc, hồ sơ cá nhân, địa chỉ nhận hàng và quyền của người dùng.
Catalog Service	Quản lý danh mục, thuộc tính, sản phẩm, biến thể, hình ảnh, giá bán và số lượng tồn kho.
Cart Service	Quản lý giỏ hàng, chương trình giảm giá, voucher và dữ liệu phục vụ quá trình kiểm tra đơn hàng.
Order Service	Quản lý phiên đặt hàng, tổng hợp thông tin mua hàng, tạo đơn và điều phối các bước giữ tồn kho, voucher và thanh toán.
Payment Service	Quản lý yêu cầu thanh toán, kết quả giao dịch, thanh toán khi nhận hàng, phí vận chuyển và dữ liệu địa giới từ GHN.
Engagement Service	Quản lý đánh giá, danh sách yêu thích và thông báo; các API nghiệp vụ của thành phần này cần được tiếp tục hoàn thiện.
PostgreSQL	Lưu trữ dữ liệu riêng của từng dịch vụ, bảo đảm mỗi nhóm dữ liệu có thành phần chịu trách nhiệm rõ ràng.
RabbitMQ	Truyền sự kiện giữa các dịch vụ đối với những nghiệp vụ có thể xử lý bất đồng bộ.
GHN và cổng thanh toán	Cung cấp các chức năng bên ngoài như tra cứu địa giới, tính phí giao hàng và xử lý giao dịch thanh toán.
AI Assistant Service	Tiếp nhận câu hỏi, truy xuất nguồn tri thức theo tenant và trả lời kèm trích dẫn; không thay thế dữ liệu nghiệp vụ của Catalog, Order hoặc Payment.

Trong môi trường phát triển, API Gateway sử dụng cổng 8080; các dịch vụ Identity, Catalog, Cart, Order, Payment và Engagement lần lượt sử dụng các cổng từ 8081 đến 8086. Thông tin cổng chỉ phục vụ việc xác định địa chỉ chạy của từng thành phần và có thể thay đổi theo môi trường triển khai.

2.3.2. Giao tiếp và triển khai các thành phần

Client và Backend được xây dựng thành các ứng dụng độc lập. Client sử dụng Next.js, trong khi Gateway và các dịch vụ Backend sử dụng Spring Boot. Hai phía có thể được biên dịch, cấu hình và triển khai riêng, miễn là Client được cung cấp đúng địa chỉ của API Gateway.

Client giao tiếp với Backend thông qua giao thức HTTP hoặc HTTPS theo kiểu REST API. Dữ liệu yêu cầu và phản hồi chủ yếu được biểu diễn dưới dạng JSON. Mọi yêu cầu từ Client được gửi đến API Gateway trước khi chuyển tới dịch vụ nghiệp vụ tương ứng.

Giữa các dịch vụ Backend có hai hình thức giao tiếp:

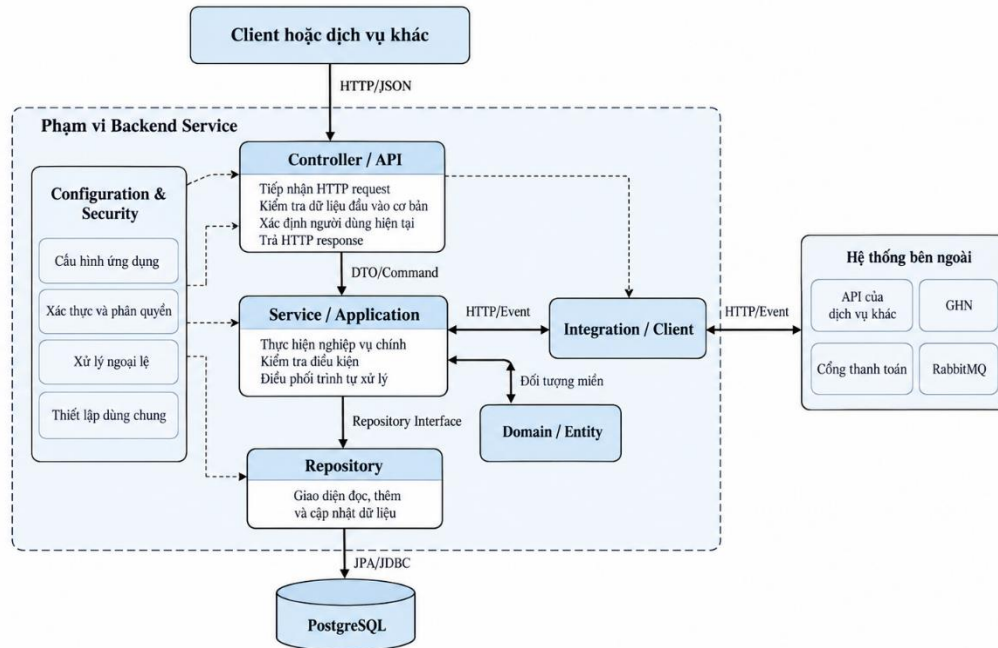
- Giao tiếp đồng bộ: được sử dụng khi dịch vụ gửi yêu cầu cần nhận kết quả ngay, chẳng hạn lấy thông tin sản phẩm, kiểm tra địa chỉ hoặc tính phí giao hàng.
- Giao tiếp bất đồng bộ: được sử dụng khi một dịch vụ chỉ cần thông báo sự kiện đã xảy ra và không cần chờ dịch vụ nhận xử lý xong. RabbitMQ đảm nhận việc truyền các sự kiện này.

Sự kết hợp giữa hai hình thức giao tiếp giúp hệ thống vừa bảo đảm tính liên tục của luồng mua hàng, vừa giảm sự phụ thuộc trực tiếp giữa các dịch vụ đối với những công việc có thể xử lý sau.

2.3.3. Tổ chức các tầng bên trong Backend

Mỗi dịch vụ Backend được tổ chức theo kiến trúc nhiều tầng. Các tầng có trách nhiệm riêng và chỉ trao đổi với nhau thông qua các đối tượng hoặc giao diện đã được xác định. Cách tổ chức này giúp tách biệt việc tiếp nhận yêu cầu, xử lý nghiệp vụ và truy cập dữ liệu.

Kiến trúc nhiều tầng của một Backend Service



Các tầng trao đổi qua DTO, đối tượng miền và giao diện đã xác định

Tầng	Trách nhiệm
Controller/API	Tiếp nhận yêu cầu HTTP, kiểm tra dữ liệu đầu vào cơ bản, xác định người dùng hiện tại và trả kết quả cho bên gọi. Tầng này không trực tiếp xử lý các quy tắc nghiệp vụ phức tạp.
Service/Application	Thực hiện nghiệp vụ chính, phối hợp nhiều thao tác và kiểm soát trình tự xử lý. Ví dụ: kiểm tra điều kiện đặt hàng, giữ tồn kho, áp dụng voucher và tạo đơn.
Domain/Entity	Biểu diễn các đối tượng và trạng thái quan trọng của nghiệp vụ như sản phẩm, giỏ hàng, đơn hàng, thanh toán và các bản ghi giữ chỗ.
Repository	Cung cấp các thao tác đọc, thêm và cập nhật dữ liệu; giúp tầng nghiệp vụ không phụ thuộc trực tiếp vào câu lệnh truy vấn cơ sở dữ liệu.
Integration/Client	Giao tiếp với dịch vụ khác, cổng thanh toán, GHN hoặc hệ thống truyền sự kiện. Tầng này chuyển đổi dữ liệu bên ngoài thành dạng mà nghiệp vụ có thể sử dụng.

Tầng	Trách nhiệm
Configuration và Security	Quản lý cấu hình ứng dụng, xác thực, phân quyền, xử lý lỗi và các thiết lập dùng chung của dịch vụ.

2.4. Thiết kế dữ liệu

DynamicMart áp dụng mô hình cơ sở dữ liệu riêng cho từng dịch vụ. Mỗi dịch vụ chịu trách nhiệm quản lý và thay đổi dữ liệu thuộc phạm vi nghiệp vụ của mình. Ví dụ, quản lý tài khoản và địa chỉ, quản lý sản phẩm và tồn kho, quản lý quá trình đặt hàng.

2.4.1. Các nhóm thực thể chính

Ba nhóm dữ liệu dưới đây tương ứng trực tiếp với ba use case trọng tâm. Tài khoản, giỏ hàng và vận chuyển được tham chiếu khi cần nhưng không mở rộng thành nhóm thiết kế riêng trong phần này.

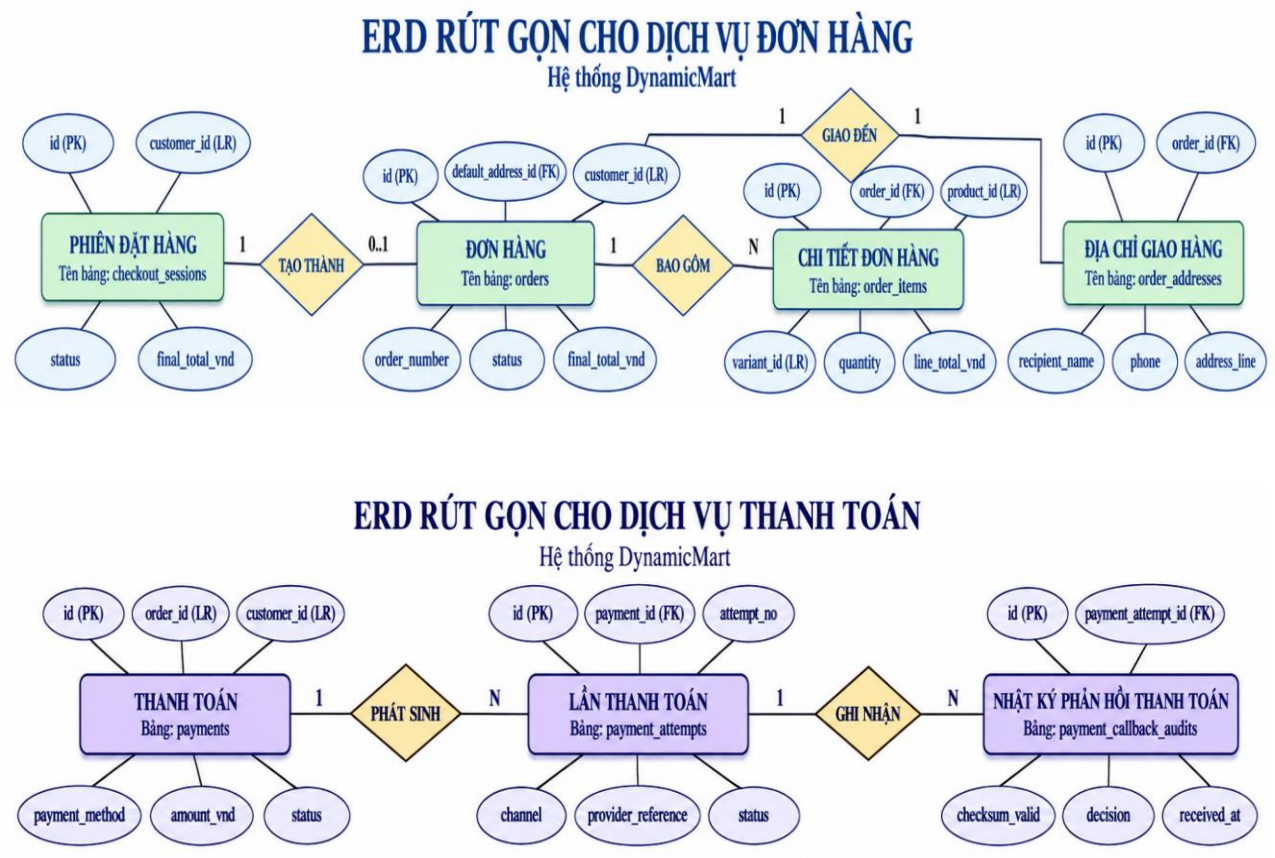
Nhóm dữ liệu	Thực thể cốt lõi	Nội dung quản lý
Danh mục sản phẩm	Category, Attribute, Product, ProductVariant, ProductImage, InventoryItem	Cấu trúc danh mục, thông tin bán, biến thể, hình ảnh và số lượng tồn kho
Đơn hàng	CheckoutSession, Order, OrderItem, OrderAddress, OrderStatusHistory	Phiên đặt hàng, bản chụp sản phẩm và địa chỉ, tổng tiền và vòng đời đơn
Thanh toán	Payment, PaymentAttempt, PaymentCallbackAudit	Nghĩa vụ thanh toán, các lần thực hiện và kết quả phản hồi

2.4.2. Các thực thể và quan hệ cốt lõi

Thực thể	Khóa a chính	Khóa ngoại hoặc tham chiếu	Quan hệ chính
categories	id	parent_id → categories.id	Một danh mục có thể có nhiều danh mục con và nhiều sản phẩm
products	id	category_id → categories.id	Một sản phẩm thuộc một danh mục và có nhiều biến thể
product_variants	id	product_id → products.id	Mỗi biến thể thuộc một sản phẩm và có tối đa một bản ghi tồn kho
inventory_items	variant_id	product_variants.id →	Lưu số lượng thực tế và số lượng đang giữ của biến thể
checkout_sessions	id	customer_id, cart_id, address_id là REF	Một phiên đặt hàng tạo tối đa một đơn hàng

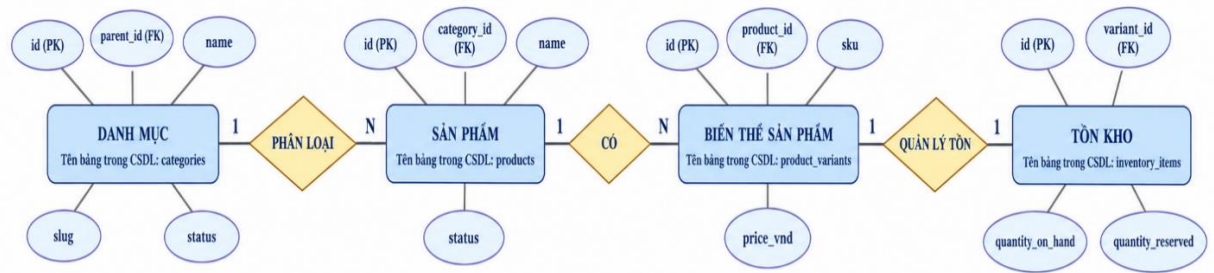
Thực thể	Khóa chính	Khóa ngoại hoặc tham chiếu	Quan hệ chính
orders	id	checkout_session_id UNIQUE; customer_id REF	Một đơn có nhiều dòng hàng, một địa chỉ và nhiều trạng thái
order_items	id	order_id → orders.id; product_id, variant_id là REF	Lưu bản chụp sản phẩm tại thời điểm đặt hàng
order_addresses	id	order_id → orders.id	Mỗi đơn lưu cố định một địa chỉ giao hàng
payments	id	order_id REF; customer_id REF	Mỗi đơn có tối đa một nghĩa vụ thanh toán
payment_attempts	id	payment_id → payments.id	Một khoản thanh toán có thể có nhiều lần thực hiện
payment_callback_audits	id	payment_attempt_id → payment_attempts.id	Lưu kết quả xác minh phản hồi thanh toán

2.4.3. Sơ đồ ERD



ERD RÚT GỌN CHO DỊCH VỤ DANH MỤC SẢN PHẨM

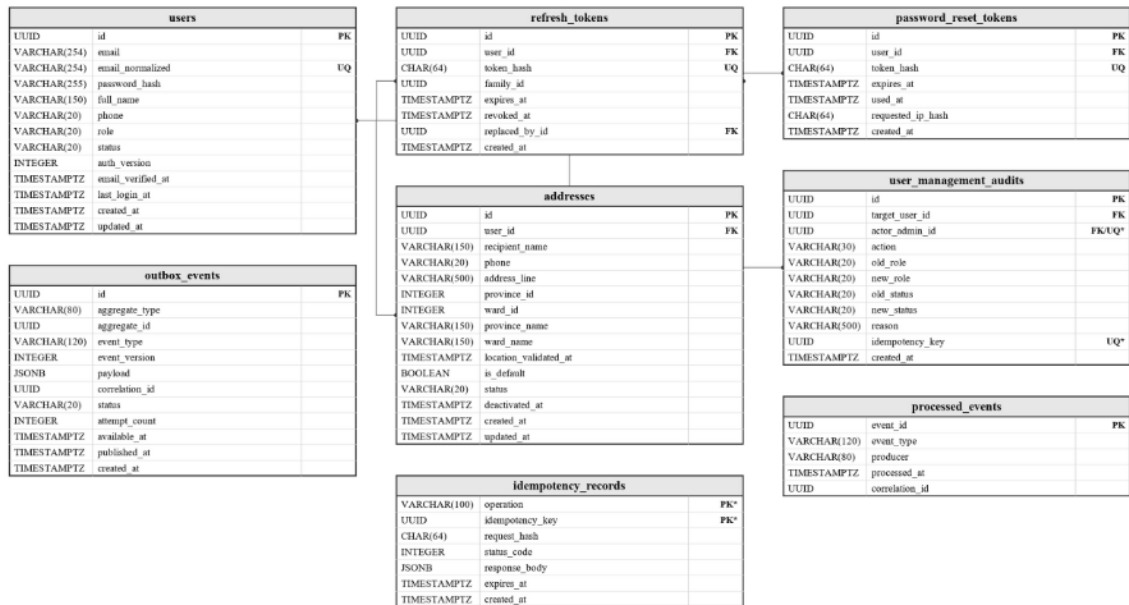
Hệ thống DynamicMart



2.4.4. Biểu đồ lớp

SƠ ĐỒ DỮ LIỆU ĐẦY ĐỦ - IDENTITY SERVICE

8 bảng dữ liệu



2.5 Thiết kế API dịch vụ

Thiết kế API bám theo ba tài nguyên nghiệp vụ chính. Các endpoint hỗ trợ được đặt trong luồng tương ứng để tránh lặp lại danh sách chức năng ở nhiều chương.

2.5.1 Quy ước API

Nội dung	Quy ước
Địa chỉ chung	http://localhost:8080/api/v1 qua API Gateway
Giao thức và dữ liệu	HTTP hoặc HTTPS; request và response sử dụng JSON
Xác thực	JWT trong Authorization; refresh token lưu bằng cookie bảo mật
Tên tài nguyên	Danh từ viết thường, số nhiều; tham số đường dẫn xác định tài nguyên
Tìm kiếm	Query parameter dùng cho từ khóa, lọc, sắp xếp và phân trang
Chống trùng gửi	Idempotency-Key bắt buộc khi tạo đơn hàng

Nội dung	Quy ước
API nội bộ	Dùng khóa nội bộ và không cho ứng dụng khách gọi trực tiếp

2.5.2 Các API chính

Use Case	Phương thức và endpoint	Dữ liệu chính	Phản hồi thành công	Lỗi chính
Danh mục sản phẩm	GET /catalog/categories	Điều kiện trạng thái	Cây danh mục	400, 503
Danh mục sản phẩm	GET /catalog/products	Từ khóa, danh mục, giá, thuộc tính, phân trang	Danh sách sản phẩm	400, 503
Danh mục sản phẩm	GET /catalog/products/{slug}	slug sản phẩm	Chi tiết, biến thể, ảnh và tồn kho	404
Hỗ trợ danh mục	POST /assistant/chat	message; tenant được Gateway cố định	Câu trả lời và nguồn trích dẫn	400, 503
Đơn hàng	POST /checkout/sessions	Giỏ hàng hoặc lựa chọn mua	Phiên đặt hàng	400, 409
Đơn hàng	POST /checkout/sessions/{id}/preview	Địa chỉ, voucher, vận chuyển	Bản xem trước tổng tiền	409, 422
Đơn hàng	POST /checkout/orders	Phiên đặt hàng; Idempotency-Key	Đơn hàng đã tạo	409, 422, 503
Đơn hàng	GET /orders/{orderId}	orderId và JWT	Chi tiết cùng lịch sử trạng thái	401, 404
Thanh toán	GET /payments/orders/{orderId}	orderId và JWT	Thông tin và trạng thái thanh toán	401, 404
Thanh toán	POST /payments/{id}/vnpay-attempt	paymentId	Đường dẫn thanh toán	409, 502
Thanh toán	POST /payments/{id}/cod-confirmations	paymentId và thông tin xác nhận	Trạng thái COD	403, 409
Thanh toán	GET /payments/return-status	Mã tham chiếu giao dịch	Kết quả đã được xác minh	400, 404

2.5.3 Phản hồi và xử lý lỗi

Phản hồi thành công trả về dữ liệu phù hợp với tài nguyên. Phản hồi lỗi gồm mã lỗi, thông báo và danh sách trường không hợp lệ khi cần. Ứng dụng khách sử dụng cả HTTP status code và mã lỗi nghiệp vụ để lựa chọn cách xử lý.

Mã trạng thái	Trường hợp sử dụng
200 hoặc 201	Truy vấn hoặc tạo dữ liệu thành công

Mã trạng thái	Trường hợp sử dụng
400	Thiếu dữ liệu, sai định dạng hoặc tham số không hợp lệ
401 hoặc 403	Chưa xác thực hoặc không có quyền
404	Không tìm thấy tài nguyên thuộc phạm vi truy cập
409 hoặc 422	Xung đột trạng thái hoặc vi phạm quy tắc nghiệp vụ
502 hoặc 503	Hệ thống tích hợp hoặc dịch vụ phụ thuộc chưa sẵn sàng

2.6 Thiết kế ứng dụng client

Ứng dụng khách được thiết kế như thành phần tiêu thụ API qua Gateway. Giao diện chỉ kiểm tra dữ liệu đầu vào cơ bản; giá, tồn kho, tổng tiền, trạng thái đơn và trạng thái thanh toán luôn được xác minh tại máy chủ.

2.6.1 Thiết kế giao diện

Use Case	Màn hình chính	Nội dung
Danh mục sản phẩm	Danh sách sản phẩm	Tìm kiếm, lọc, sắp xếp và phân trang
Danh mục sản phẩm	Chi tiết sản phẩm	Hình ảnh, thuộc tính, biến thể, giá và khả năng mua
Danh mục sản phẩm	Quản trị danh mục và tồn kho	Danh mục, sản phẩm, biến thể, trạng thái và số lượng tồn
Danh mục sản phẩm	Trợ lý mua sắm	Gửi câu hỏi, nhận câu trả lời và nguồn trích dẫn
Đơn hàng	Xác nhận đặt hàng	Địa chỉ, voucher, vận chuyển, phương thức và bản xem trước
Đơn hàng	Lịch sử và chi tiết đơn	Mã đơn, dòng hàng, tổng tiền và lịch sử trạng thái
Đơn hàng	Quản trị đơn hàng	Tra cứu và chuyển trạng thái theo quy tắc
Thanh toán	Lựa chọn thanh toán	COD hoặc VNPay theo điều kiện của đơn
Thanh toán	Kết quả và quản trị giao dịch	Số tiền, phương thức, lần thực hiện và trạng thái

2.6.2 Thiết kế tích hợp API

Use Case	API sử dụng	Cách sử dụng trên giao diện
Danh mục sản phẩm	GET /catalog/categories; GET /catalog/products; GET /catalog/products/{slug}; POST /assistant/chat	Tải danh mục và sản phẩm; hiển thị chi tiết; gửi câu hỏi và trình bày nguồn tham khảo

Use Case	API sử dụng	Cách sử dụng trên giao diện
Đơn hàng	POST /checkout/sessions; POST /checkout/sessions/{id}/preview; POST /checkout/orders; GET /orders/{id}	Chuẩn bị dữ liệu, tính lại tổng tiền, tạo đơn và hiển thị lịch sử
Thanh toán	GET /payments/orders/{orderId}; POST /payments/{id}/vnpay-attempt; POST /payments/{id}/cod-confirmations; GET /payments/return-status	Khởi tạo phương thức phù hợp và xác minh kết quả trước khi hiển thị

2.6.3 Luồng dữ liệu và trạng thái

Thao tác trên giao diện tạo request qua lớp dùng chung, sau đó gửi đến API Gateway. Response được ánh xạ thành dữ liệu hiển thị. Mỗi màn hình phân biệt trạng thái đang tải, thành công, không có dữ liệu, lỗi nhập liệu, lỗi nghiệp vụ và lỗi kết nối. Trợ lý mua sắm hiển thị nguồn trích dẫn và thông báo chưa đủ thông tin khi không tìm thấy dữ liệu phù hợp.

CHƯƠNG 3. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

3.1 Môi trường và công cụ phát triển

DynamicMart được phát triển trên môi trường Windows. Backend được tổ chức thành các ứng dụng Spring Boot độc lập, trong khi Client được xây dựng bằng Next.js và TypeScript. Mã nguồn được quản lý bằng Git, các thư viện được quản lý lần lượt bằng Maven và npm.

3.1.1. Môi trường phát triển

Thành phần	Công nghệ hoặc công cụ	Phiên bản	Mục đích sử dụng
Hệ điều hành	Windows	10	Phát triển và chạy thử
Dịch vụ Java	Java; Spring Boot	17.0.12; 4.1.1	Xây dựng Gateway và sáu dịch vụ nghiệp vụ
Hạ tầng dịch vụ	Spring Cloud; Maven	2025.1.3; 3.9.16	Định tuyến, quản lý phụ thuộc và build
Dữ liệu nghiệp vụ	PostgreSQL; Spring Data JPA; Flyway	13 trở lên	Lưu trữ, truy cập và quản lý migration
Sự kiện	RabbitMQ	Theo cấu hình	Trao đổi sự kiện bất đồng bộ
Ứng dụng khách	Node.js; Next.js; React; TypeScript	22.19.0; 16.3.5; 19.2.8; 5.x	Xây dựng giao diện và tích hợp API
Trợ lý AI	Python; FastAPI; Uvicorn	3.12; 0.115.12; 0.34.3	Cung cấp API hỏi đáp và điều phối RAG
Mô hình ngôn ngữ	Ollama	qwen2.5:0.5b	Sinh câu trả lời tiếng Việt trong môi trường cục bộ
Mô hình embedding	Ollama	nomic-embed-text	Biểu diễn nội dung tri thức thành vector
Cơ sở dữ liệu vector	Qdrant	qdrant-client 1.19.1	Lưu vector và truy xuất theo tenant
Kiểm thử	JUnit 5; Spring Boot Test; unittest; Postman	Theo dự án	Kiểm thử dịch vụ Java, trợ lý AI và hợp đồng API
Quản lý mã nguồn	Git	2.43.0	Theo dõi thay đổi
Đóng gói	Docker; Docker Compose	29.7.2; 5.3.1	Khởi tạo cơ sở dữ liệu, RabbitMQ và Qdrant

3.1.2. Môi trường phát triển backend

Phần máy chủ gồm API Gateway, sáu dịch vụ nghiệp vụ và một dịch vụ trợ lý AI:

Thành phần	Vai trò
API Gateway	Tiếp nhận request, kiểm tra JWT, CORS và định tuyến
Identity Service	Xác thực, hồ sơ, địa chỉ và quản lý người dùng
Catalog Service	Danh mục, thuộc tính, sản phẩm, biến thể và tồn kho
Cart Service	Giỏ hàng, giảm giá trực tiếp và voucher
Order Service	Phiên thanh toán, tạo đơn và điều phối nghiệp vụ
Payment Service	Thanh toán, phí vận chuyển và dữ liệu địa giới
Engagement Service	Đánh giá, yêu thích, hỏi đáp, thông báo và tương tác
AI Assistant Service	Hỏi đáp dựa trên RAG, trả lời kèm nguồn và tách dữ liệu theo tenant

3.1.3. Công cụ kiểm thử

Công cụ	Cách sử dụng
JUnit 5	Kiểm thử các quy tắc và phương thức nghiệp vụ
Spring Boot Test	Kiểm thử Controller, Service và cấu hình bảo mật
Maven Test	Chạy toàn bộ bộ kiểm thử của từng dịch vụ
Postman	Kiểm tra endpoint, request body, header và status code
Swagger	Quan sát API trong quá trình phát triển
PostgreSQL	Kiểm tra migration, ràng buộc và dữ liệu thực
Trình duyệt	Kiểm thử giao diện, điều hướng và tích hợp API
ESLint	Kiểm tra mã nguồn TypeScript và React

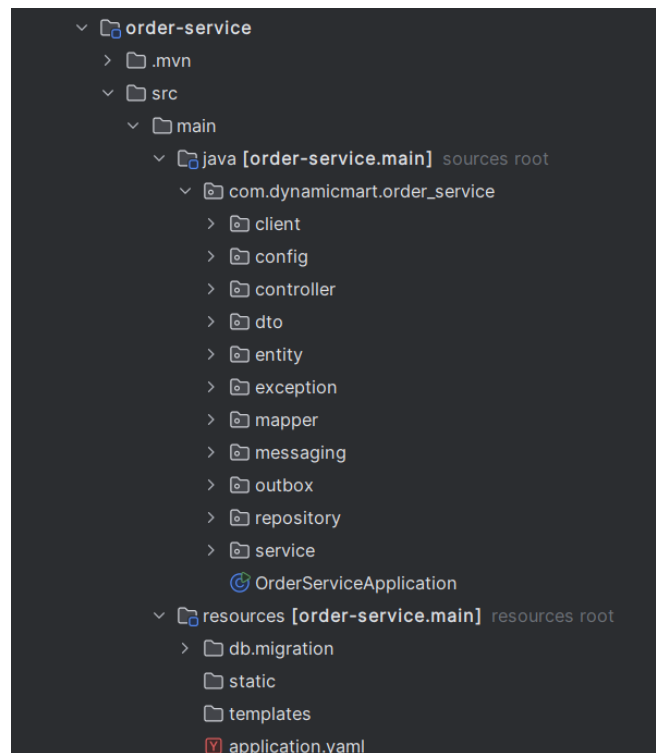
3.2 Cài đặt Backend Service API

Backend của DynamicMart được hiện thực theo kiến trúc hướng dịch vụ. API Gateway là điểm tiếp nhận yêu cầu chung, trong khi mỗi nhóm nghiệp vụ được triển khai thành một ứng dụng Spring Boot độc lập và sở hữu cơ sở dữ liệu riêng.

3.2.1 Cấu trúc dự án Backend

Kho mã nguồn đặt API Gateway ở thư mục api-gateway và sáu dịch vụ trong services. Mỗi module Spring Boot có pom.xml, Maven Wrapper, mã nguồn chính, test và application.yaml. Bên trong dịch vụ, mã được chia thành controller hoặc api, service hoặc application, repository, entity, dto, mapper, messaging, outbox, config và exception.

[CHÈN HÌNH 3.1: Ảnh cây thư mục dự án trong IDE, chỉ mở các module và package chính]



Vai trò của các package chính được thể hiện trong bảng sau:

Package	Vai trò trong mã nguồn
controller	Tiếp nhận HTTP request, đọc tham số, kiểm tra dữ liệu cơ bản và trả response
service	Thực hiện các quy tắc và luồng xử lý nghiệp vụ
repository	Truy vấn và lưu dữ liệu bằng Spring Data JPA
entity	Ánh xạ đối tượng nghiệp vụ với bảng trong PostgreSQL
dto.request	Khai báo cấu trúc dữ liệu Client gửi lên
dto.response	Khai báo cấu trúc dữ liệu API trả về
config	Cấu hình bảo mật, thời gian, HTTP Client và các thành phần dùng chung
exception	Khai báo ngoại lệ nghiệp vụ và chuẩn hóa phản hồi lỗi
client	Gọi API nội bộ của dịch vụ khác
query	Thực hiện các truy vấn tìm kiếm, lọc và phân trang phức tạp

mapper	Chuyển Entity thành DTO và ngược lại
messaging	Phát hoặc tiếp nhận sự kiện giữa các dịch vụ
outbox	Lưu sự kiện chờ phát cùng giao dịch nghiệp vụ

3.2.2 Hiện thực dữ liệu và truy cập cơ sở dữ liệu

Các entity dùng JPA để ánh xạ bảng; repository kế thừa Spring Data và bổ sung truy vấn cần thiết. Flyway chạy migration khi dịch vụ khởi động. Tại thời điểm đọc mã nguồn, Identity và Engagement có migration khởi tạo; Cart và Catalog có migration bổ sung; Order có năm migration nguồn; Payment có sáu migration nguồn. Việc tách migration theo service ngăn một module thay đổi lược đồ của module khác.

Những thao tác dễ cạnh tranh như giữ tồn kho dùng bản ghi reservation và log thao tác. Payment lưu riêng payment, payment attempt và callback audit. Order lưu snapshot thay vì phụ thuộc vào dữ liệu sống ở Catalog hay Identity khi hiển thị lịch sử.

a) Mô hình sản phẩm và tồn kho

Ba Entity tiêu biểu của Catalog Service gồm:

Entity	Bảng dữ liệu	Nội dung quản lý
Product	products	Thông tin chung, danh mục, trạng thái và thời điểm công bố
ProductVariant	product_variants	SKU, giá, kích thước, khối lượng và trạng thái biến thể
InventoryItem	inventory_items	Số lượng thực tế, số lượng đang giữ và số lượng khả dụng

Entity này cũng cung cấp các phương thức reserve, commit và release. Nhờ đó, quy tắc tồn kho được đặt ngay trong đối tượng nghiệp vụ thay vì phân tán tại Controller.

Các Entity dễ xảy ra cập nhật đồng thời như ProductVariant, InventoryItem, CheckoutSession và CustomerOrder sử dụng trường @Version. Trường này giúp phát hiện trường hợp hai yêu cầu cùng cập nhật một bản ghi cũ.

b) Mô hình đơn hàng

Khi đơn hàng được tạo, Order Service lưu dữ liệu dưới dạng bản chụp. Các thông tin sản phẩm, giá, địa chỉ, voucher và vận chuyển không phụ thuộc vào dữ liệu có thể thay đổi sau này.

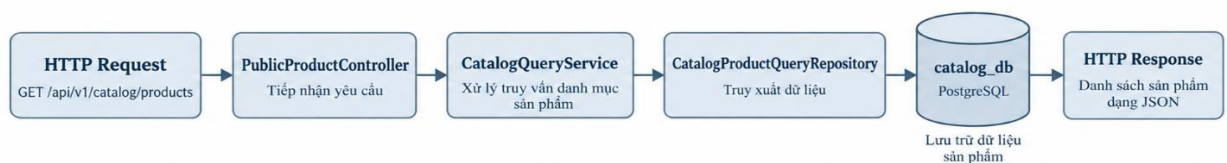
Entity	Vai trò
CustomerOrder	Lưu thông tin chung, tổng tiền, thanh toán và trạng thái đơn
OrderItem	Lưu bản chụp sản phẩm, biến thể, giá và số lượng
OrderAddress	Lưu địa chỉ giao hàng tại thời điểm đặt
OrderVoucherSnapshot	Lưu voucher và số tiền giảm
OrderShippingSnapshot	Lưu phí, dịch vụ và thời gian giao dự kiến
OrderStatusHistory	Lưu lịch sử thay đổi trạng thái
OrderSaga	Theo dõi tiến trình và kết quả của quy trình tạo đơn

3.2.3 Hiện thực API chính

a) API tìm kiếm sản phẩm

API tìm kiếm sản phẩm được hiện thực trong `PublicProductController`. Controller tiếp nhận từ khóa, danh mục, khoảng giá, thuộc tính lọc, cách sắp xếp và thông tin phân trang. Các tham số này được chuyển thành đối tượng `ProductSearchCriteria` trước khi gửi đến `CatalogQueryService`.

Luồng lấy danh sách sản phẩm



`CatalogQueryService` kiểm tra tính hợp lệ của điều kiện tìm kiếm, chẳng hạn khoảng giá không được âm, giá tối thiểu không được lớn hơn giá tối đa và kích thước trang không được vượt quá giới hạn. Sau đó, `CatalogProductQueryRepository` thực hiện truy vấn kết hợp theo các điều kiện đã nhận.

Kết quả truy vấn được chuyển thành `ProductSummaryResponse`, chỉ chứa những dữ liệu cần thiết cho màn hình danh sách như tên sản phẩm, hình ảnh, giá bán, giá khuyến

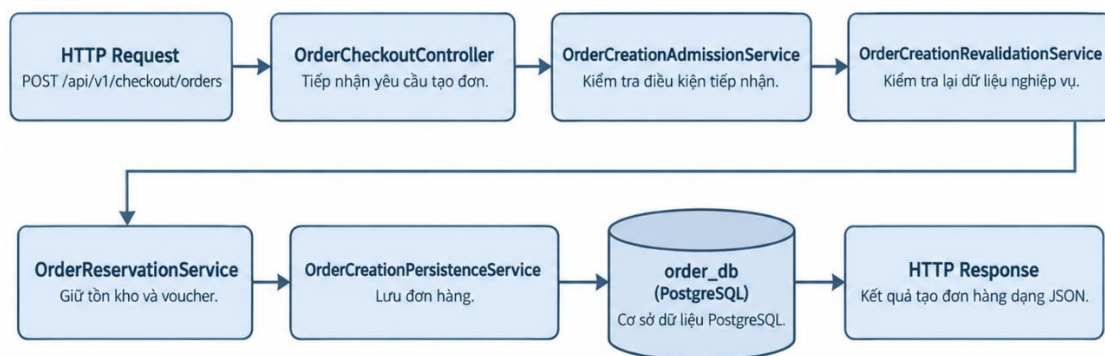
mại và trạng thái còn hàng. Việc sử dụng DTO giúp API không trả trực tiếp Entity của cơ sở dữ liệu.

Trợ lý mua sắm được tích hợp qua POST /api/v1/assistant/chat. Gateway chuyển yêu cầu đến dịch vụ FastAPI; nội dung được chia đoạn, tạo embedding, truy xuất trong Qdrant theo tenant và đưa vào Ollama để tạo câu trả lời kèm nguồn. Bản hiện tại chủ yếu sử dụng kho tri thức đã kiểm duyệt và chưa coi chatbot là nguồn dữ liệu giả hoặc tồn kho thời gian thực.

b) API tạo đơn hàng

API tạo đơn hàng là nghiệp vụ trung tâm của hệ thống và được hiện thực trong OrderCheckoutController. Do quá trình tạo đơn liên quan đến nhiều dịch vụ, phần xử lý được chia thành các Service nhỏ thay vì đặt toàn bộ logic trong Controller.

Luồng tạo đơn hàng



Đầu tiên, OrderCreationAdmissionService kiểm tra phiên thanh toán và Idempotency-Key. Tiếp theo, OrderCreationRevalidationService kiểm tra lại địa chỉ, mặt hàng, giá, voucher, phí vận chuyển và phương thức thanh toán.

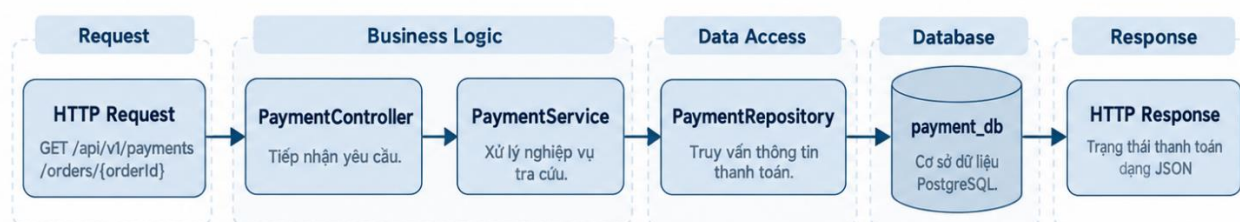
Khi các điều kiện hợp lệ, OrderReservationService thực hiện giữ tồn kho và lượt sử dụng voucher. OrderCreationPersistenceService sau đó lưu đơn hàng, các dòng hàng và bản chụp địa chỉ, voucher, vận chuyển. Kết quả trả về gồm mã đơn, số đơn, trạng thái và đường dẫn thanh toán nếu khách hàng chọn thanh toán trực tuyến.

Việc chia nhỏ quá trình tạo đơn giúp từng bước có trách nhiệm rõ ràng và thuận lợi hơn khi xử lý bù trừ nếu một dịch vụ gặp lỗi.

c) API kiểm tra trạng thái thanh toán

Sau khi khách hàng hoàn tất thao tác tại cổng thanh toán, Client gọi API kiểm tra trạng thái giao dịch. API này được hiện thực trong PaymentController, còn việc truy vấn và xác định kết quả được thực hiện tại PaymentService.

Luồng tra cứu trạng thái thanh toán



PaymentService kiểm tra giao dịch có thuộc đơn hàng hợp lệ hay không và trả về các dữ liệu chính như số tiền, phương thức, thời điểm thanh toán và trạng thái hiện tại.

Client không tự kết luận giao dịch thành công từ tham số trên URL. Trạng thái chỉ được hiển thị sau khi dữ liệu thanh toán đã được backend xác minh và lưu vào cơ sở dữ liệu.

3.2.4 Hiện thực xử lý nghiệp vụ

Logic nghiệp vụ của DynamicMart được đặt chủ yếu tại các lớp Service và một số phương thức trong Entity. Controller chỉ tiếp nhận request, lấy thông tin xác thực và chuyển dữ liệu đến Service tương ứng. Cách tổ chức này giúp backend không trở thành tập hợp các API CRUD đơn thuần.

Một số nghiệp vụ tiêu biểu được hiện thực như sau:

Nghệp vụ	Lớp xử lý chính	Điều kiện được kiểm tra
Tìm kiếm sản phẩm	CatalogQueryService	Khoảng giá, thuộc tính lọc, phân trang và trạng thái sản phẩm
Giữ và trừ tồn kho	InventoryReservationService	Số lượng khả dụng, trạng thái reservation và thao tác lặp
Tạo đơn hàng	Các Service thuộc Order Service	Phiên thanh toán, địa chỉ, giá, voucher, vận chuyển và thanh toán
Xử lý thanh toán	PaymentService	Số tiền, mã tham chiếu, chữ ký và trạng thái giao dịch
Hỏi đáp về sản phẩm	RagService	Tenant, nguồn truy xuất, ngưỡng liên quan và tính có căn cứ của dữ liệu số

a) Kiểm tra tính hợp lệ của dữ liệu

Các DTO request sử dụng Jakarta Validation để kiểm tra những điều kiện cơ bản như trường bắt buộc, định dạng email, độ dài chuỗi và giới hạn số lượng. Việc kiểm tra được thực hiện trước khi dữ liệu đi vào lớp Service.

Các điều kiện mang tính nghiệp vụ được kiểm tra tại Service. Ví dụ, số lượng sản phẩm có thể đúng về kiểu dữ liệu nhưng vẫn bị từ chối nếu vượt quá tồn kho. Tương tự, voucher có thể tồn tại nhưng không được sử dụng nếu hết hạn hoặc không đạt giá trị đơn hàng tối thiểu.

b) Kiểm soát quyền sở hữu dữ liệu

Mã người dùng được lấy từ JWT thay vì nhận trực tiếp từ request body. Khi truy vấn giỏ hàng, địa chỉ hoặc đơn hàng, Repository luôn kết hợp mã tài nguyên với mã khách hàng.

Nhờ đó, một khách hàng không thể thay đổi customerId trong request để truy cập dữ liệu của khách hàng khác. Trường hợp không tìm thấy tài nguyên thuộc quyền sở hữu được phản hồi bằng 404 Not Found hoặc 403 Forbidden tùy theo nghiệp vụ.

c) Kiểm soát dữ liệu đồng thời

Các dữ liệu có khả năng được cập nhật cùng lúc như tồn kho, mặt hàng trong giỏ và đơn hàng sử dụng trường phiên bản. Khi Client gửi một phiên bản cũ, hệ thống trả về 409 Conflict và yêu cầu tải lại dữ liệu mới.

Đối với những thao tác quan trọng như giữ tồn kho hoặc tạo đơn, Repository sử dụng khóa cơ sở dữ liệu trong phạm vi transaction. Cách xử lý này hạn chế việc nhiều yêu cầu đồng thời làm bán vượt tồn kho hoặc tạo nhiều đơn từ cùng một phiên thanh toán.

d) Chống xử lý trùng lặp

API tạo đơn và các thao tác điều chỉnh quan trọng sử dụng Idempotency-Key. Nếu Client gửi lại cùng một yêu cầu do mất kết nối, hệ thống trả lại kết quả đã xử lý thay vì tạo thêm dữ liệu mới.

Nếu một khóa đã được dùng cho nội dung khác, hệ thống từ chối yêu cầu và trả về 409 Conflict. Quy tắc này đặc biệt quan trọng trong quá trình tạo đơn, giữ tồn kho và xác nhận thanh toán.

e) Xử lý giao dịch nhiều dịch vụ

Quá trình tạo đơn liên quan đến Order Service, Catalog Service, Cart Service và Payment Service. Hệ thống sử dụng Saga để theo dõi từng bước xử lý.

Nếu giữ tồn kho thành công nhưng giữ voucher thất bại, phần tồn kho đã giữ được hoàn lại. Nếu thanh toán trả trước thất bại hoặc hết hạn, các tài nguyên đang giữ cũng được giải phóng theo trạng thái của đơn hàng.

Cơ chế này giúp hệ thống duy trì tính nhất quán mà không cần cho phép một dịch vụ truy cập trực tiếp cơ sở dữ liệu của dịch vụ khác.

f) Xử lý ngoại lệ

Mỗi dịch vụ có lớp `GlobalExceptionHandler` để chuyển ngoại lệ thành phản hồi HTTP thống nhất. Lỗi trả về gồm mã lỗi, thông báo, danh sách trường không hợp lệ và thời điểm phát sinh.

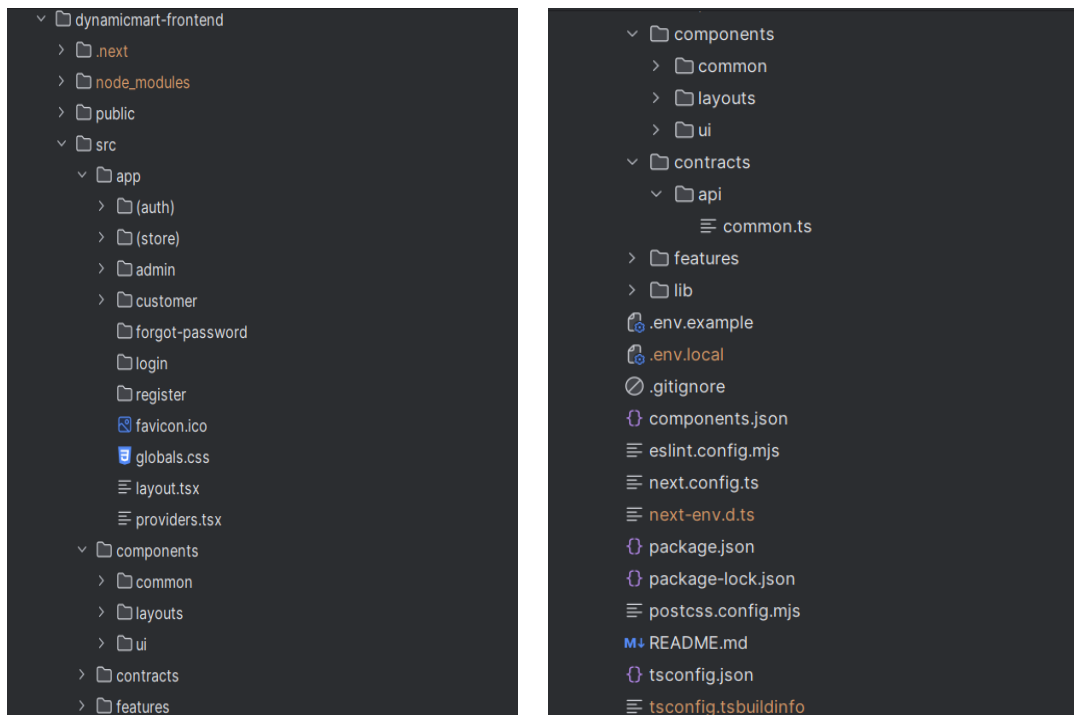
Mã lỗi giúp Client lựa chọn cách xử lý, còn thông báo được sử dụng để trình bày cho người dùng. Các lỗi dữ liệu đầu vào thường trả về 400, lỗi xác thực trả về 401, lỗi không có quyền trả về 403, tài nguyên không tồn tại trả về 404, còn xung đột nghiệp vụ trả về 409 hoặc 422.

3.3 Cài đặt Client Application

3.3.1 Cấu trúc dự án Client

Ứng dụng Client được hiện thực bằng Next.js, React và TypeScript. Client không chỉ đảm nhiệm việc hiển thị giao diện mà còn là thành phần tiêu thụ các dịch vụ do backend cung cấp.

Các chức năng được chia thành từng module nghiệp vụ. Mỗi module chứa component giao diện, kiểu dữ liệu và lớp gọi API riêng. Cách tổ chức này giúp tách phần trình bày khỏi phần giao tiếp với backend.



Vai trò của các thư mục chính:

Thư mục	Vai trò
app	Khai báo các trang, layout và cấu trúc điều hướng
components/ui	Chứa các thành phần giao diện dùng chung như nút, ô nhập và danh sách chọn
components/layouts	Chứa bố cục cửa hàng, tài khoản và quản trị
features	Tổ chức mã nguồn theo từng nhóm nghiệp vụ
features/*/api	Chứa các hàm gọi API của từng chức năng
features/*/components	Chứa component giao diện thuộc từng chức năng
features/*/types	Khai báo kiểu dữ liệu request và response
lib/api	Cung cấp cơ chế gọi API dùng chung
lib/auth	Quản lý access token và trạng thái phiên đăng nhập
contracts	Khai báo cấu trúc phản hồi dùng chung giữa Client và backend

Việc chia project theo feature giúp mã nguồn của sản phẩm, giỏ hàng, khách hàng và thanh toán không bị trộn lẫn trong một thư mục component chung.

3.3.2 Giao diện và chức năng chính

Các màn hình được nhóm theo ba use case trọng tâm. Những chức năng hỗ trợ chỉ xuất hiện tại bước cần sử dụng, nhờ đó cấu trúc giao diện thống nhất với phần phân tích ở Chương 2.

Use Case	Màn hình hiện thực	Chức năng chính	API tiêu biểu
Danh mục sản phẩm	Danh sách, chi tiết, quản trị danh mục và tồn kho, trợ lý mua sắm	Tìm kiếm, lọc, chọn biến thể, quản lý dữ liệu bán và hỏi đáp có nguồn	Catalog API; POST /api/v1/assistant/chat
Đơn hàng	Xác nhận đặt hàng, lịch sử, chi tiết và quản trị đơn	Tạo bản xem trước, tạo đơn bằng khóa chống gửi trùng và theo dõi trạng thái	Checkout API; Order API
Thanh toán	Lựa chọn phương thức, chuyển hướng VNPay, kết quả và quản trị giao dịch	Khởi tạo COD hoặc VNPay, kiểm tra và hiển thị trạng thái đã xác minh	Payment API

Trợ lý mua sắm được gắn trong StoreLayout nên có thể mở tại các trang cửa hàng. Widget quản lý lịch sử câu hỏi trong phiên giao diện, hiển thị câu trả lời và danh sách nguồn. Khi dịch vụ không phản hồi, giao diện giữ nội dung hiện tại và cho phép gửi lại.

Trong luồng Đơn hàng, mọi thay đổi về địa chỉ, voucher hoặc vận chuyển đều yêu cầu tạo lại bản xem trước. Khi tạo đơn, Idempotency-Key được giữ nguyên nếu request phải gửi lại do mất kết nối.

Trong luồng Thanh toán, giao diện không suy luận kết quả từ URL trả về. Trạng thái chỉ được hiển thị sau khi Payment API xác minh và lưu kết quả.

3.3.3 Tích hợp Client với Backend API

Mỗi feature chứa kiểu dữ liệu, hàm gọi API và component riêng. apiClient gắn địa chỉ Gateway, thông tin xác thực và chuẩn hóa lỗi trước khi trả kết quả cho giao diện.

Use Case	Thao tác	API chính	Dữ liệu hiển thị
Danh mục sản phẩm	Duyệt và tìm kiếm	GET /api/v1/catalog/products	Danh sách, giá, hình ảnh và tình trạng bán
Danh mục sản phẩm	Xem chi tiết	GET /api/v1/catalog/products/{slug}	Biến thể, thuộc tính, giá và tồn kho
Danh mục sản phẩm	Hỏi trợ lý	POST /api/v1/assistant/chat	Câu trả lời, ý định và nguồn trích dẫn
Đơn hàng	Tạo bản xem trước	POST /api/v1/checkout/sessions/{id}/preview	Mặt hàng, giảm giá, phí và tổng tiền

Use Case	Thao tác	API chính	Dữ liệu hiển thị
Đơn hàng	Tạo đơn	POST /api/v1/checkout/orders	Mã đơn, trạng thái và liên kết thanh toán
Đơn hàng	Xem chi tiết	GET /api/v1/orders/{orderId}	Bản chụp đơn và lịch sử trạng thái
Thanh toán	Khởi tạo VNPAY	POST /api/v1/payments/{id}/vnpay-attempt	Đường dẫn thanh toán
Thanh toán	Xác nhận COD	POST /api/v1/payments/{id}/cod-confirmations	Trạng thái thu tiền
Thanh toán	Kiểm tra kết quả	GET /api/v1/payments/return-status	Trạng thái giao dịch đã xác minh

Dữ liệu phản hồi được chuyển thành state của màn hình. Component không truy cập trực tiếp từng dịch vụ và không tự tính các giá trị quyết định nghiệp vụ.

3.3.4 Xử lý trạng thái và lỗi

Mỗi màn hình phân biệt trạng thái đang tải, thành công, không có dữ liệu, lỗi nhập liệu, lỗi nghiệp vụ, lỗi xác thực và lỗi kết nối.

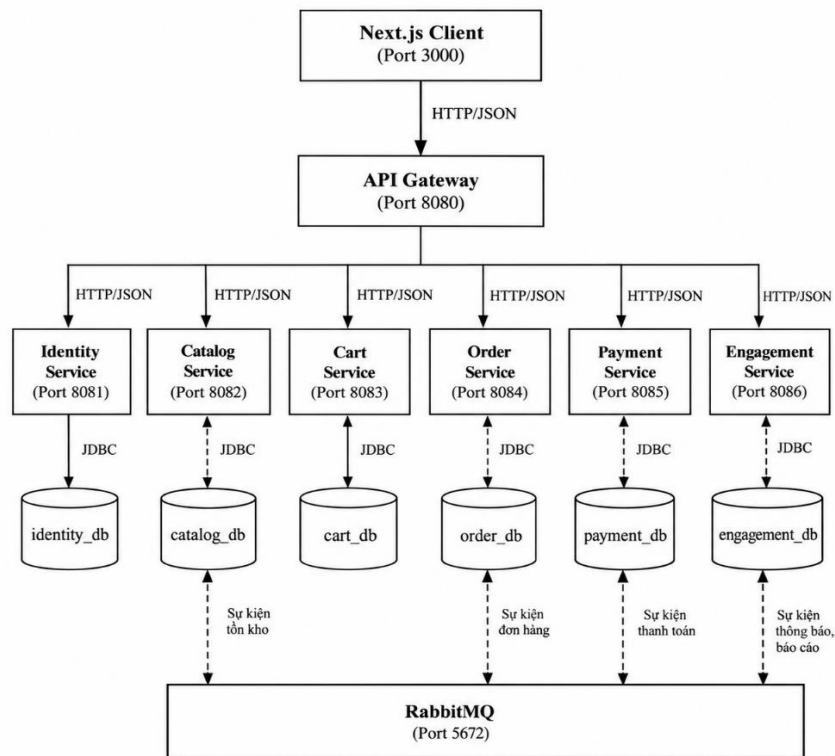
Phạm vi	Cách xử lý
Danh mục sản phẩm	Giữ điều kiện tìm kiếm khi tải lại; hiển thị trạng thái trống; yêu cầu chọn lại biến thể khi dữ liệu bán thay đổi.
Trợ lý mua sắm	Khóa thao tác khi đang gửi; hiển thị nguồn; thông báo chưa đủ dữ liệu hoặc cho phép thử lại khi mất kết nối.
Đơn hàng	Khóa nút xác nhận trong khi xử lý; yêu cầu tạo lại bản xem trước khi giá, tồn kho, voucher hoặc báo giá thay đổi.
Thanh toán	Bắt đầu ở trạng thái đang kiểm tra; chỉ chuyển sang thành công, thất bại, hết hạn hoặc đang chờ sau khi nhận kết quả từ API.

3.4 Triển khai hệ thống

Trong phạm vi hiện tại, DynamicMart được triển khai trên môi trường cục bộ. Ứng dụng khách, Gateway, các dịch vụ nghiệp vụ, trợ lý AI, PostgreSQL, RabbitMQ, Qdrant và Ollama được khởi động độc lập rồi kết nối thông qua cấu hình môi trường.

3.4.1. Mô hình triển khai cục bộ

Mô hình triển khai gồm các thành phần sau:



[CHÈN HÌNH MÔ HÌNH TRIỂN KHAI]

Ứng dụng khách chỉ gửi request đến API Gateway. Gateway xác thực và chuyển request đến dịch vụ phù hợp, bao gồm endpoint trợ lý AI. Mỗi dịch vụ nghiệp vụ chỉ truy cập cơ sở dữ liệu thuộc phạm vi của mình; trợ lý AI truy xuất Qdrant theo tenant và gọi Ollama để tạo câu trả lời.

3.4.2. Thứ tự khởi động

Các thành phần được khởi động theo trình tự sau:

1. Khởi động PostgreSQL.
2. Tạo sáu cơ sở dữ liệu của các dịch vụ.
3. Khởi động RabbitMQ nếu kiểm thử luồng sự kiện.
4. Khởi động Identity, Catalog, Cart và Payment Service.
5. Khởi động Order Service sau khi các dịch vụ phụ thuộc đã sẵn sàng.
6. Khởi động Engagement Service.

Khởi động Qdrant, Ollama và AI Assistant Service khi kiểm thử chức năng trợ lý.

7. Khởi động API Gateway.
8. Khởi động ứng dụng Next.js Client.

Cơ sở dữ liệu và các dịch vụ nền cần được khởi động trước để Flyway có thể kiểm tra migration và các lời gọi liên dịch vụ không bị gián đoạn.

3.4.3. Công chạy các thành phần

Thành phần	Cổng mặc định	Vai trò
Next.js Client	3000	Cung cấp giao diện web
API Gateway	8080	Tiếp nhận và định tuyến request
Identity Service	8081	Xác thực và tài khoản
Catalog Service	8082	Sản phẩm, biên thẻ và tồn kho
Cart Service	8083	Giỏ hàng và voucher
Order Service	8084	Checkout và đơn hàng
Payment Service	8085	Thanh toán và vận chuyển
Engagement Service	8086	Đánh giá và trưng tác
PostgreSQL	5432	Lưu trữ dữ liệu
RabbitMQ	5672	Trao đổi sự kiện
RabbitMQ Management	15672	Theo dõi hàng đợi và kết nối
AI Assistant Service	8001	Cung cấp API hỏi đáp RAG
Qdrant	6333	Lưu trữ và truy xuất vector
Ollama	11434	Cung cấp embedding và mô hình hội thoại

Khi cổng 8080 đã được sử dụng trên máy phát triển, Gateway có thể được chuyển sang cổng 18080. Khi đó, biến NEXT_PUBLIC_API_BASE_URL của Client phải được cập nhật tương ứng.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Kiểm thử API

Kiểm thử tập trung vào ba use case đã xác định ở Chương 2. Mỗi nhóm bao gồm trường hợp hợp lệ, dữ liệu không hợp lệ và lỗi nghiệp vụ; kết quả được đối chiếu qua response, trạng thái lưu trữ và giao diện.

4.1.1 Kiểm thử Danh mục sản phẩm

ã	API hoặc chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	ết quả K
C01	GET /catalog/products	Điều kiện lọc hợp lệ	Trả đúng danh sách và phân trang	Đạt
C02	GET /catalog/products	Khoảng giá không hợp lệ	Trả lỗi dữ liệu đầu vào	Đạt
C03	Kiểm tra biến thể	Sản phẩm ngừng bán hoặc hết tồn	Không cho phép tiếp tục mua	Đạt
C04	POST /assistant/chat	Câu hỏi có nguồn phù hợp	Trả lời tiếng Việt kèm trích dẫn	Đạt
C05	POST /assistant/chat	Câu hỏi không có nguồn	Thông báo chưa đủ thông tin, không suy đoán	Đạt

4.1.2 Kiểm thử Đơn hàng

ã	API hoặc chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	ết quả K
C06	POST /checkout/sessions	Mặt hàng và tài khoản hợp lệ	Tạo phiên đặt hàng	Đạt
C07	POST /checkout/orders	Phiên hợp lệ và khóa mới	Tạo đúng một đơn hàng	Đạt
C08	POST /checkout/orders	Gửi lại cùng Idempotency-Key	Trả kết quả cũ, không tạo đơn mới	Đạt
C09	POST /checkout/orders	Giá, tồn kho hoặc phiên đã thay đổi	Từ chối và yêu cầu tạo lại bản xem trước	Đạt
C10	Luồng tạo đơn	Giữ tồn thành công nhưng bước sau thất bại	Hoàn lại tài nguyên đã giữ	Đạt

4.1.3 Kiểm thử Thanh toán

ã	API hoặc chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	ết quả K
C11	Xác nhận COD	Thông tin thu tiền hợp lệ	Ghi nhận trạng thái COD	Đạt
C12	Khởi tạo VNPAY	Khoản thanh toán hợp lệ	Trả đường dẫn thanh toán	Đạt

ã	API hoặc chức năng	Dữ liệu kiểm thử	Kết quả mong đợi	ết quả K
C13	Xử lý phản hồi VNPAY	Sai chữ ký hoặc sai số tiền	Không cập nhật thành công	Đạt
C14	Xử lý phản hồi VNPAY	Phản hồi đã xử lý	Không tạo tác động lần hai	Đạt
C15	GET /payments/return-status	Mã tham chiếu hợp lệ	Trả đúng trạng thái đã xác minh	Đạt

4.2 Kiểm thử tích hợp Client API

Kiểm thử tích hợp xác nhận giao diện gửi đúng request qua Gateway, chuyển response thành dữ liệu hiển thị và xử lý được trạng thái lỗi của ba luồng chính.

ã	Use Case	Luồng kiểm thử	Kết quả mong đợi	ết quả K
T01	Danh mục sản phẩm	Tìm kiếm, lọc và mở chi tiết	Dữ liệu hiển thị đúng theo Catalog API	Đạt
T02	Danh mục sản phẩm	Chọn biến thể	Giá, hình ảnh và tồn kho thay đổi theo response	Đạt
T03	Danh mục sản phẩm	Đặt câu hỏi cho trợ lý	Hiển thị câu trả lời và nguồn; xử lý được lỗi kết nối	Đạt
T04	Đơn hàng	Tạo bản xem trước và xác nhận đơn	Tổng tiền được tính lại và đơn chỉ tạo một lần	Đạt
T05	Đơn hàng	Xem chi tiết và lịch sử trạng thái	Hiển thị đúng bản chụp của đơn	Đạt
T06	Thanh toán	Chọn VNPAY và quay lại hệ thống	Chuyển hướng đúng và kiểm tra kết quả qua API	Đạt
T07	Thanh toán	Kiểm tra COD hoặc giao dịch đang chờ	Hiển thị đúng trạng thái do Payment API trả về	Đạt

4.3 Đánh giá mức độ đáp ứng

Mức độ đáp ứng được đối chiếu trực tiếp với ba use case trọng tâm và các yêu cầu kỹ thuật cần thiết để vận hành chúng.

Use Case	Mức độ thực hiện	Kết quả chính
Danh mục sản phẩm	Hoàn thành phần chính	Có API và giao diện cho danh mục, sản phẩm, biến thể, tồn kho; trợ lý RAG đã tích hợp ở mức hỗ trợ hỏi đáp theo nguồn.
Đơn hàng	Hoàn thành phần chính	Có phiên đặt hàng, bản xem trước, chống gửi trùng, giữ tài nguyên, tạo đơn và lưu bản chụp.
Thanh toán	Hoàn thành phần chính	Hỗ trợ COD, khởi tạo VNPAY, xác minh phản hồi và tra cứu trạng thái theo đơn.

Đánh giá yêu cầu kỹ thuật:

Yêu cầu	Mức độ thực hiện	Ghi chú
Kiến trúc hướng dịch vụ	Hoàn thành	Các phạm vi nghiệp vụ có dịch vụ và dữ liệu riêng
REST API và API Gateway	Hoàn thành phần chính	Có định tuyến, xác thực và hợp đồng JSON
Tích hợp ứng dụng khách	Hoàn thành phần chính	Ba use case chính đã kết nối với API
Nhất quán liên dịch vụ	Hoàn thành phần chính	Có reservation, Saga và idempotency
Trợ lý AI RAG	Hoàn thành bản thử nghiệm	Có FastAPI, Qdrant, Ollama, phân tách tenant và trích dẫn nguồn
Đóng gói và triển khai	Hoàn thành một phần	Hạ tầng hỗ trợ Docker Compose; toàn hệ thống vẫn chạy cục bộ
Giám sát và kiểm thử tải	Chưa hoàn thiện	Cần bổ sung trước khi triển khai thực tế

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

DynamicMart đã hiện thực được nền tảng thương mại điện tử theo kiến trúc hướng dịch vụ, trong đó ứng dụng khách giao tiếp với các dịch vụ qua API Gateway và mỗi dịch vụ quản lý dữ liệu thuộc phạm vi của mình.

Kết quả được tập trung vào ba use case: Danh mục sản phẩm, Đơn hàng và Thanh toán. Các luồng chính đã kết nối từ giao diện đến API, xử lý quy tắc nghiệp vụ và lưu trữ dữ liệu. Trợ lý AI RAG được bổ sung để hỗ trợ hỏi đáp theo nguồn tri thức đã kiểm duyệt, nhưng không thay thế các API quyết định giá, tồn kho, trạng thái đơn hoặc thanh toán.

2. Hạn chế

Hệ thống chủ yếu được kiểm thử trong môi trường cục bộ. Một số giao diện quản trị, cơ chế theo dõi vận hành và tình huống lỗi liên dịch vụ vẫn cần hoàn thiện. Trợ lý AI chưa được cấp dữ liệu Catalog thời gian thực nên chỉ phù hợp với câu hỏi dựa trên kho tri thức hiện có.

3. Hướng phát triển

Hướng phát triển tập trung vào hoàn thiện ba luồng nghiệp vụ, bổ sung kiểm thử tải và giám sát tập trung, chuẩn hóa Docker Compose và triển khai trên hạ tầng thực tế. Trợ lý AI có thể được mở rộng bằng bộ chuyển đổi phía máy chủ để nhận dữ liệu sản phẩm đã xác minh từ Catalog Service, đồng thời tiếp tục duy trì giới hạn bảo mật đối với tài khoản, đơn hàng và thanh toán.

TÀI LIỆU THAM KHẢO

- [1] Tài liệu hướng dẫn viết báo cáo môn Phát triển phần mềm hướng dịch vụ.
- [2] Kho mã nguồn DynamicMart, README.md và thư mục docs, truy cập nội bộ ngày 29/09/2026.
- [3] Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/>.
- [4] Spring Security Reference, <https://docs.spring.io/spring-security/reference/>.
- [5] Spring Cloud Stream Reference, <https://docs.spring.io/spring-cloud-stream/reference/>.
- [6] Next.js Documentation, <https://nextjs.org/docs>.
- [7] PostgreSQL Documentation, <https://www.postgresql.org/docs/>.
- [8] RabbitMQ Documentation, <https://www.rabbitmq.com/docs>.
- [9] Flyway Documentation, <https://documentation.red-gate.com/flyway>.